



北京航空航天大学
BEIHANG UNIVERSITY



数据结构与程序设计（信息类）

Data Structure & Programming

北京航空航天大学 数据结构课程组

软件学院 谭火彬

2024年春



提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题

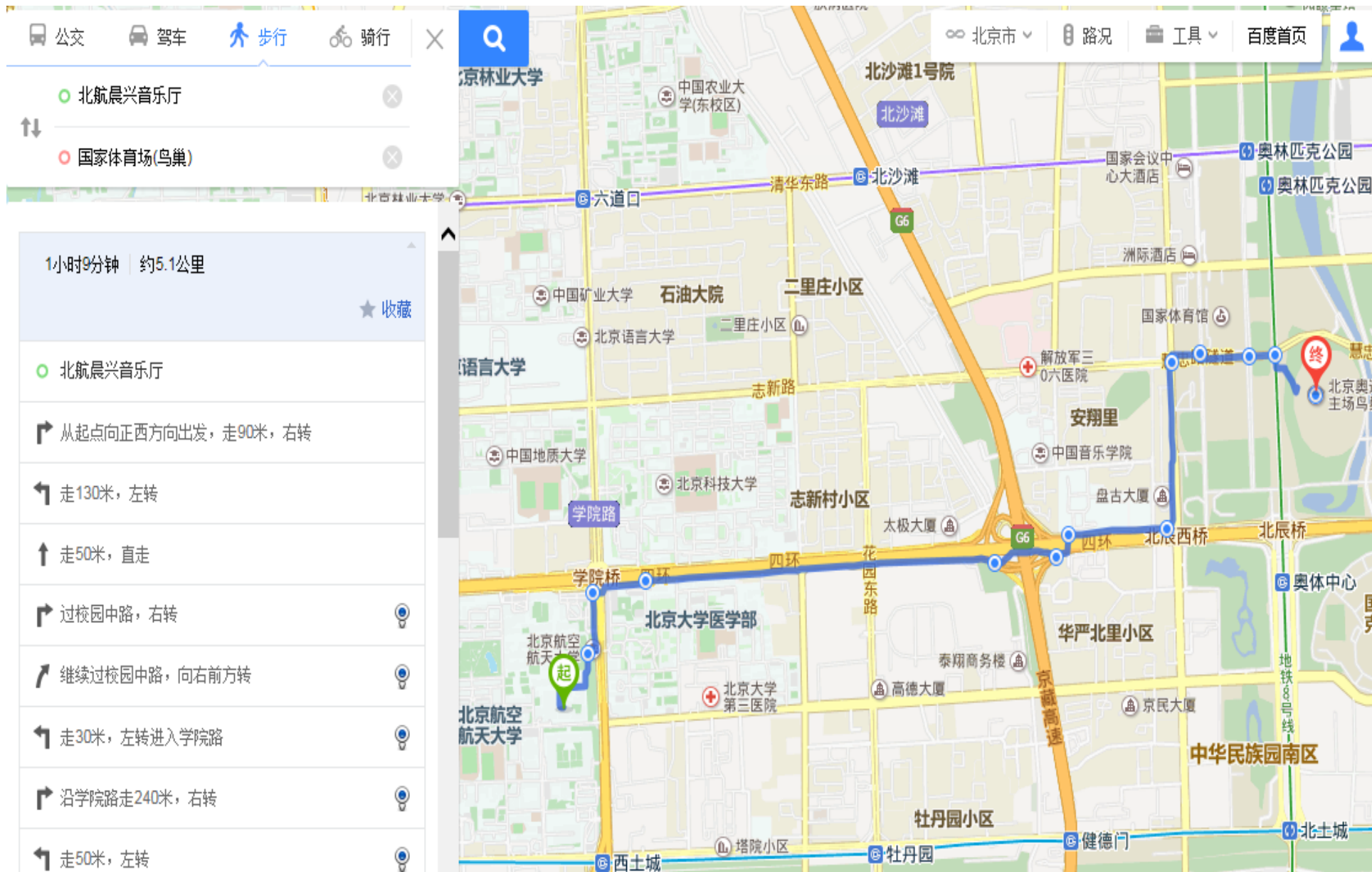


提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题



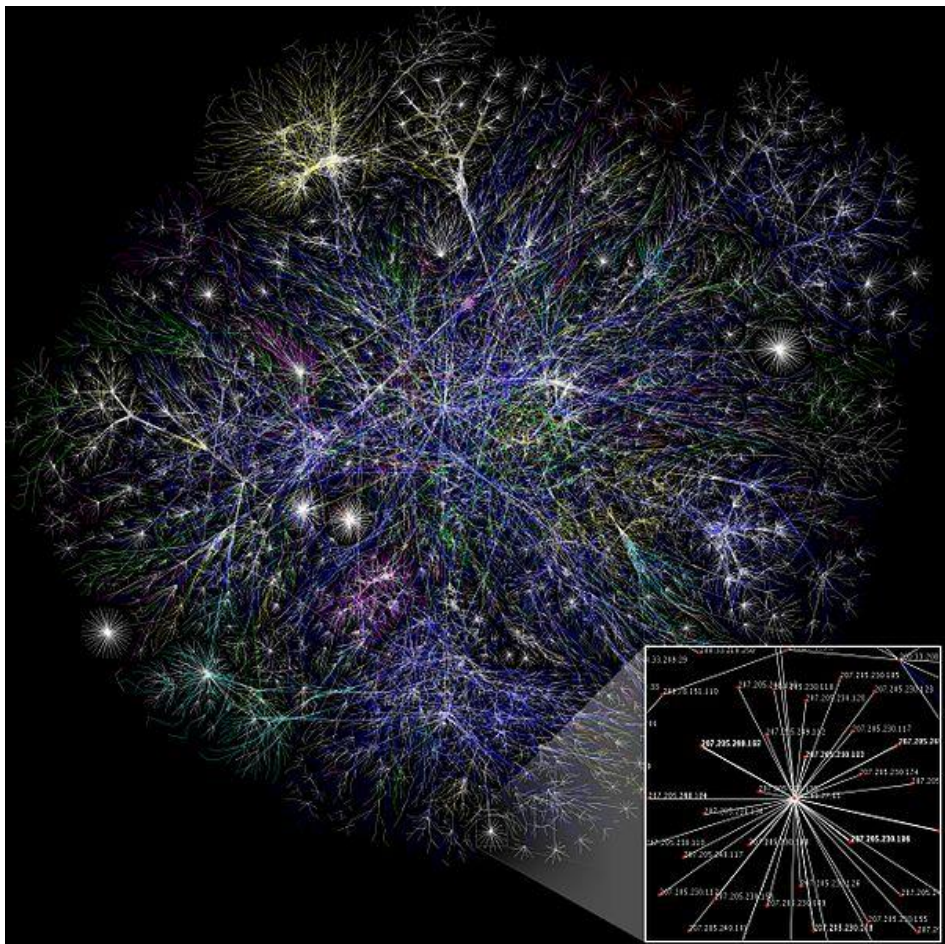
图：地图导航





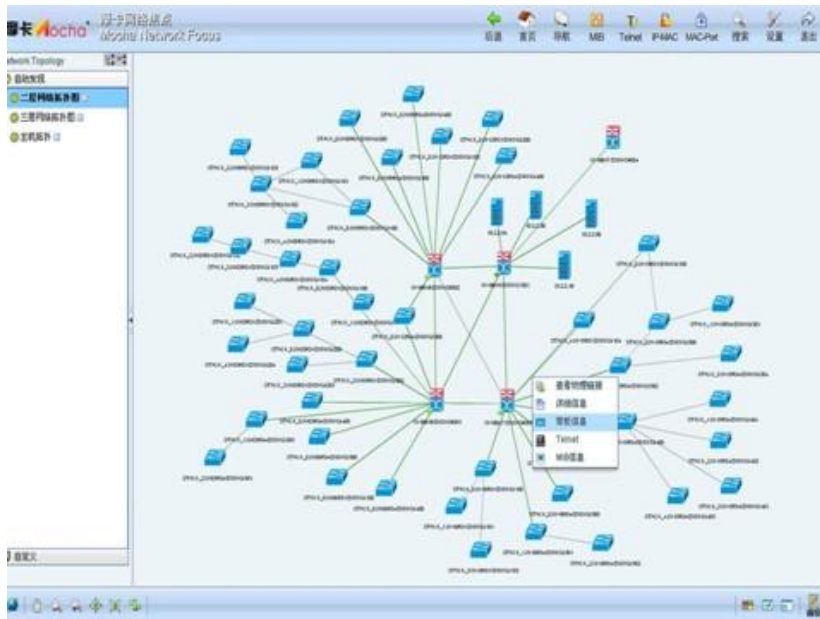
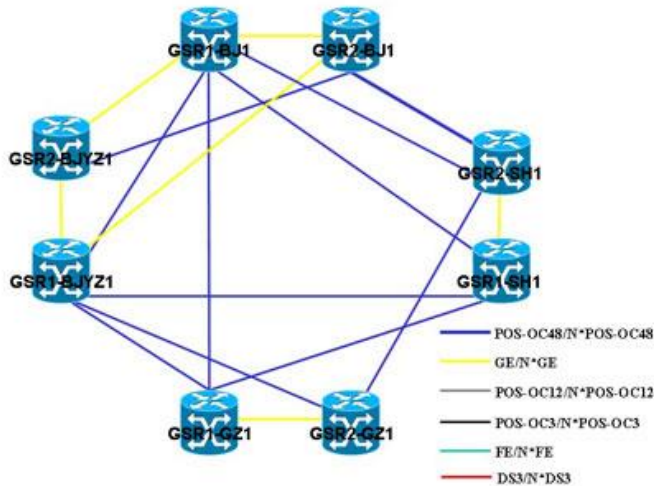
图：网络拓扑

Internet Map



计算机网络

CNCNET骨干网超级核心节点之间互连结构图





图：社交网络 (Social Network)

Facebook helps you connect and share with the people in your life.





地铁线路图





问题7.1：北京地铁乘坐线路查询

- ◆编写一个程序实现北京地铁乘坐线路查询，输入为起始站名和终点站名，输出为从起始到终点的换乘线路。
- ◆要求给出：
 - ✓乘坐站数最少的换乘方式（理论最快，通常用于计算票价），如从西土城到北京西站：
 - 西土城-10-黄庄-4-国家图书馆-9-北京西站
 - ✓换乘次数最少的换乘方式（最方便），如从西土城到北京西站：
 - 西土城-10-六里桥-9-北京西站



7.1 图 (Graph) 的基本概念

◆ 图的定义:

图是由顶点的非空有穷集合与顶点之间关系(边或弧)的集合构成的结构, 通常表示为: $G = (V, E)$

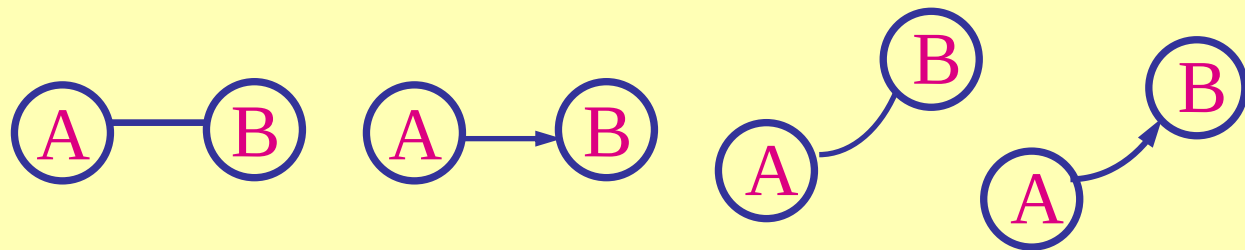
其中, V 为顶点集合, E 为关系(边或弧)的集合。



图中的顶点和关系(边)

一个数据元素---- 顶点(Vertex) 

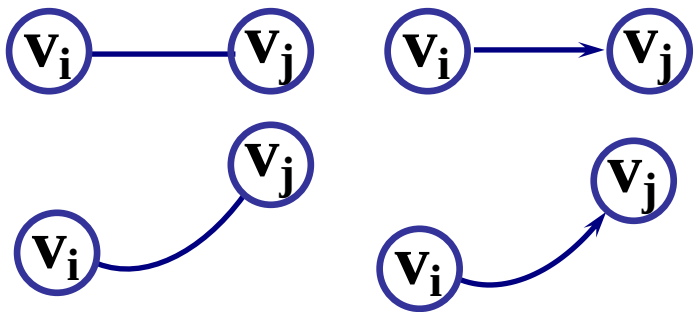
数据元素(顶点)之间的关系---- 边(弧,Edge)





边 (弧) 的表示

(1) 用图形



(2) 用符号

(v_i, v_j) 或 $\langle v_i, v_j \rangle$

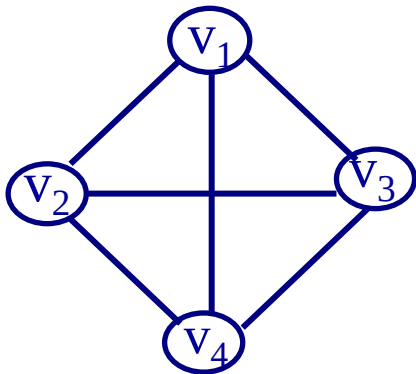
顶点偶对

(3) 用语言

- 顶点 v_i 与 v_j 是这条边的两个邻接点
- 这条边依附于顶点 v_i 和顶点 v_j



示例

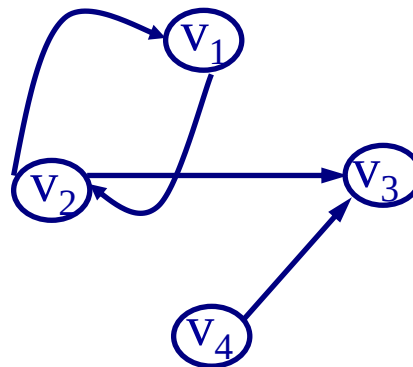


$$G_1 = (V_1, E_1)$$

其中

$$V_1 = \{ v_1, v_2, v_3, v_4 \}$$

$$E_1 = \{ (v_1, v_2), (v_1, v_3), (v_1, v_4), (v_2, v_3), (v_2, v_4), (v_3, v_4) \}$$



$$G_2 = (V_2, E_2)$$

其中

$$V_2 = \{ v_1, v_2, v_3, v_4 \}$$

$$E_2 = \{ \langle v_1, v_2 \rangle, \langle v_2, v_1 \rangle, \langle v_2, v_3 \rangle, \langle v_4, v_3 \rangle \}$$



二. 图的分类

无向图

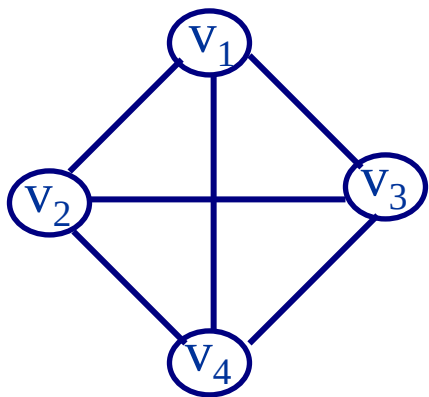
对于 $(v_i, v_j) \in E$, 必有 $(v_j, v_i) \in E$, 并且偶对中顶点的前后顺序无关

有向图

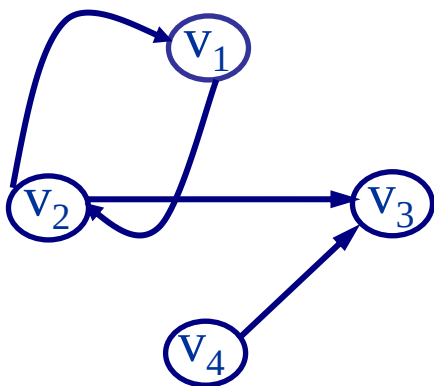
若 $\langle v_i, v_j \rangle \in E$ 是顶点的有序偶对

网(络)

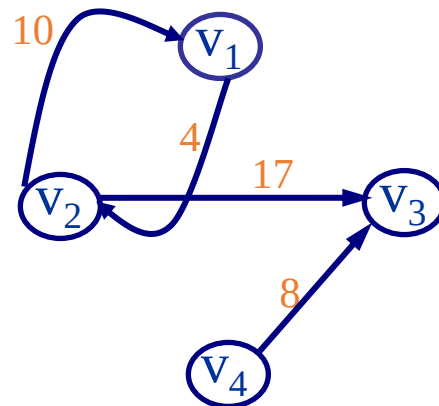
与边有关的数据称为 **权**, 边上带权的图称为 **网络**



无向图



有向图



网(络)



三. 名词术语

1. 顶点的度

依附于顶点 v_i 的边的数目,记为 $TD(v_i)$

对于有向图而言, 有:

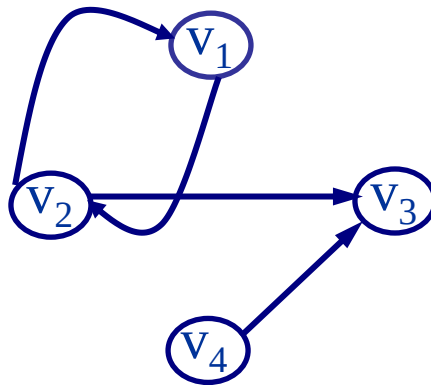
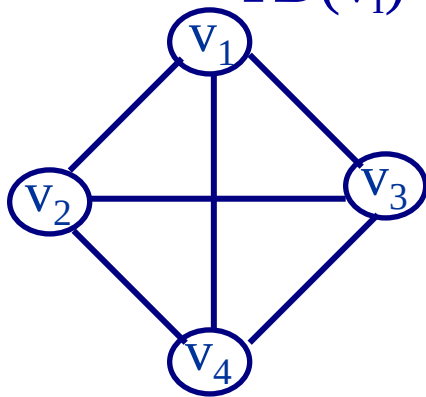
顶点的 **出度**:

以顶点 v_i 为出发点的边的数目,记为 $OD(v_i)$ 。

顶点的 **入度**:

以顶点 v_i 为终止点的边的数目,记为 $ID(v_i)$ 。

$$TD(v_i) = OD(v_i) + ID(v_i)$$





有关度与边的性质

结论1

对于具有 n 个顶点, e 条边的图,有

$$e = \frac{1}{2} \sum_{i=1}^n TD(v_i)$$

结论2

具有 n 个顶点的无向图最多有 $n(n-1)/2$ 条边

结论3

具有 n 个顶点的有向图最多有 $n(n-1)$ 条边

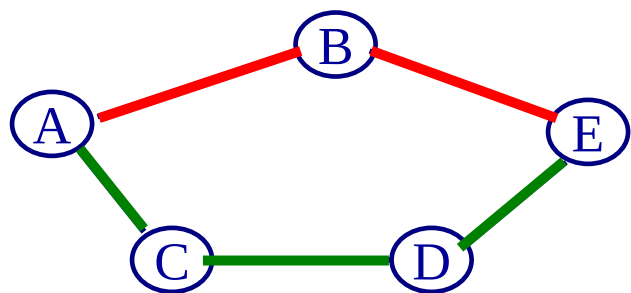
边的数目达到最大的图称为 **完全图**。
边的数目达到或接近最大的图称为 **稠密图**，
否则，称为 **稀疏图**。



2. 路径和路径长度

$(v_x, v_{i1}), (v_{i1}, v_{i2}), \dots, (v_{im}, v_y)$ 或
 $\langle v_x, v_{i1} \rangle, \langle v_{i1}, v_{i2} \rangle, \dots, \langle v_{im}, v_y \rangle$ 都在 E 中

顶点 v_x 到 v_y 之间有路径 $P(v_x, v_y)$ 的充分必要条件为:
存在顶点序列 $v_x, v_{i1}, v_{i2}, \dots, v_{im}, v_y$, 并且 序列中相邻两个顶点构成的顶点偶对分别为图中的一条边



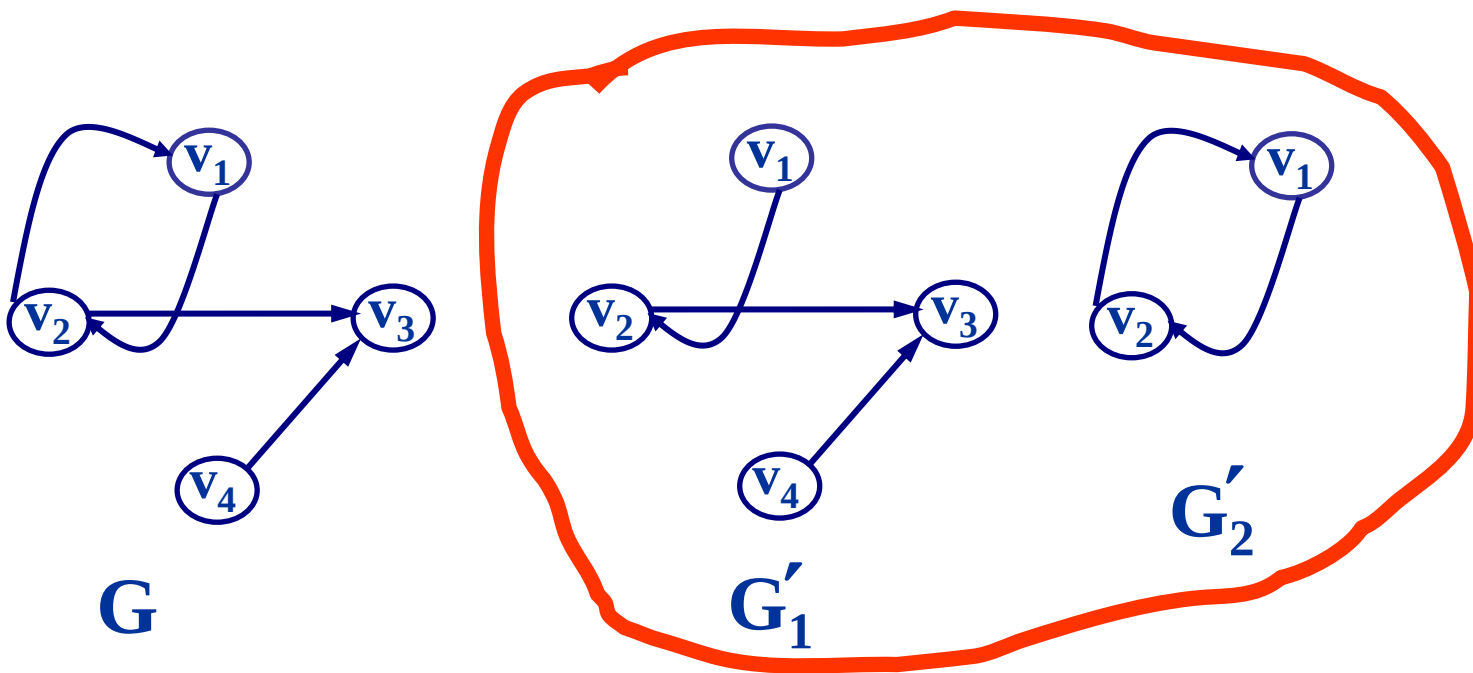
$P(A, E)$: A, B, E (第1条路径)
 A, C, D, E (第2条路径)

出发点与终止点相同的路径称为回路或环；顶点序列中顶点不重复出现的路径称为简单路径。不带权的图的路径长度是指路径上所经过的边的数目，带权图的路径长度是指路径上经过的边上的权值之和



3. 子图

对于图 $G=(V,E)$ 与 $G'=(V',E')$, 若有 $V'\subseteq V, E'\subseteq E$, 则称 G' 为 G 的一个子图





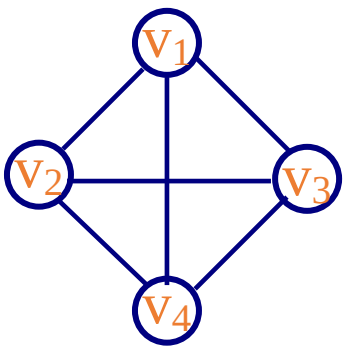
4. 图的连通 (Connected)

(1) 无向图(Digraph)的连通

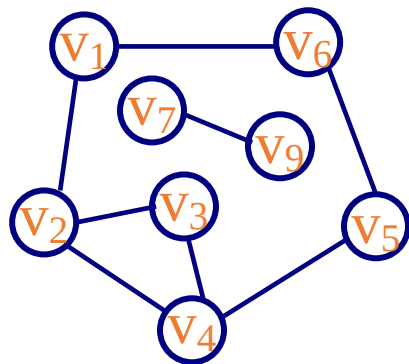
无向图中顶点 v_i 到 v_j 有路径,则称顶点 v_i 与 v_j 是连通的。若无向图中任意两个顶点都连通,则称该无向图是连通的。

连通分量

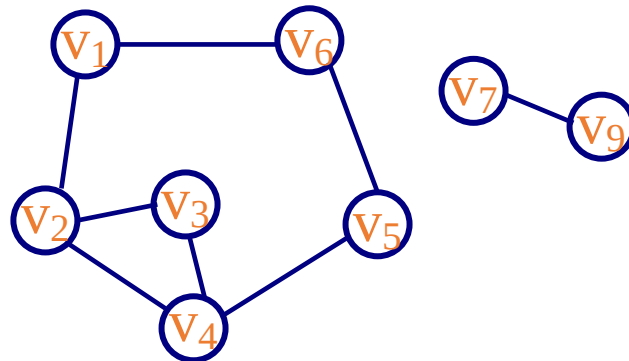
—— 无向图中的极大连通子图。



连通图



非连通图



2个连通分量



4. 图的连通(续)

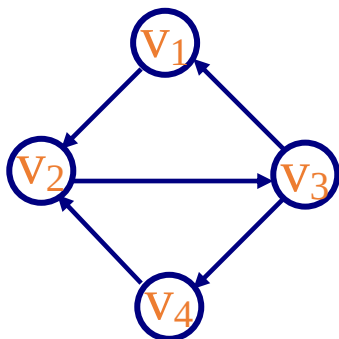
(2) 有向图(Directed Graph)的连通

若有向图中顶点 v_i 到 v_j 有路径,并且顶点 v_j 到 v_i 也有路径,则称顶点 v_i 与 v_j 是连通的

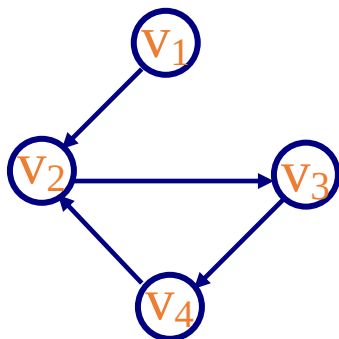
若有向图中任意两个顶点都连通,则称该有向图是强连通的

强连通分量

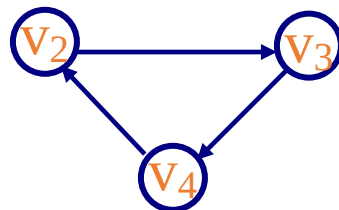
—— 有向图中的极大强连通子图



(强)连通图



非连通图



强连通分量





5. 生成树 (续)

包含具有 n 个顶点的连通图 G 的全部 n 个顶点,仅包含其 $n-1$ 条边的极小连通子图称为 G 的一个生成树

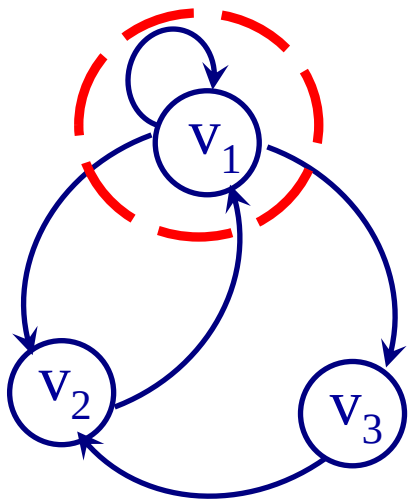
性质:

1. 包含 n 个顶点的图: **连通**且仅有 $n-1$ 条边
 $\Leftrightarrow$ **无回路**且仅有 $n-1$ 条边
 $\Leftrightarrow$ 无回路且连通
 $\Leftrightarrow$ 是一棵树
2. 如果 n 个顶点的图中只要少于 $n-1$ 条边, 图将不连通
3. 如果 n 个顶点的图中有多于 $n-1$ 条边, 图将有环 (回路)
4. 一般情况下, 生成树不唯一



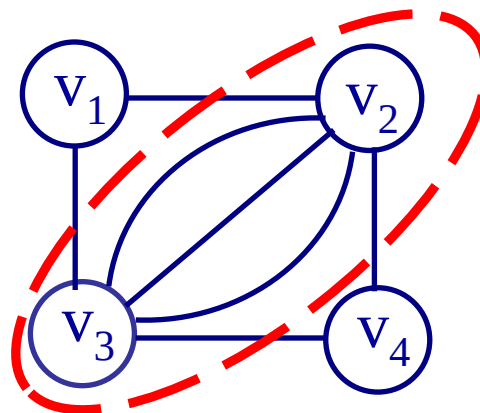
本章不讨论的图

1. 带自身环的图



2. 多重图

(作业中有一个题)



只讨论简单图



提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题



7.2 图的存储方法

- ◆ 对于一个图，需要存储的信息应该包括
 - ✓ (1) 所有顶点的数据信息
 - ✓ (2) 顶点之间关系（边或弧）的信息
 - ✓ (3) 权的信息（对于网络）



一. 邻接矩阵存储方法

◆核心思想：采用两个数组存储一个图

- 顶点信息
- 边或弧的信息
- 权

1. 定义一个一维数组VERTEX[0..n-1]存放图中所有顶点的数据信息(若顶点信息为0,1,2,3,...,此数组可略)。
2. 定义一个二维数组A[0..n-1,0..n-1]存放图中所有顶点之间关系的信息(该数组被称为邻接矩阵),有

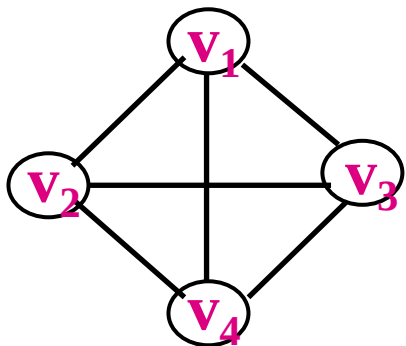
$$A[i][j]=\begin{cases} 1 & \text{当顶点}v_i\text{到顶点}v_j\text{有边时} \\ 0 & \text{当顶点}v_i\text{到顶点}v_j\text{无边时} \end{cases}$$

对于带权的图 (网、网络) , 有

$$A[i][j]=\begin{cases} w_{ij} & \text{当顶点}v_i\text{到}v_j\text{有边,且边的权为}w_{ij} \\ \infty & \text{当顶点}v_i\text{到顶点}v_j\text{无边时 (权值不存在的值)} \end{cases}$$



例

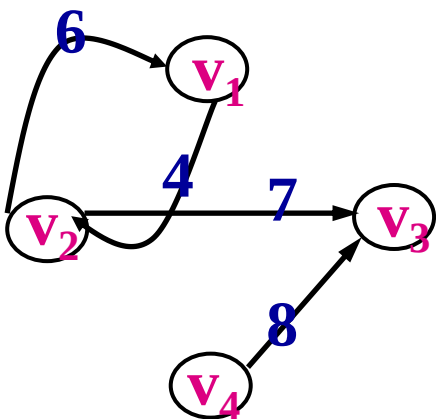


VERTEX1[0..3]

0 v_1
1 v_2
2 v_3
3 v_4

$$A_1 = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

邻接矩阵



VERTEX2[0..3]

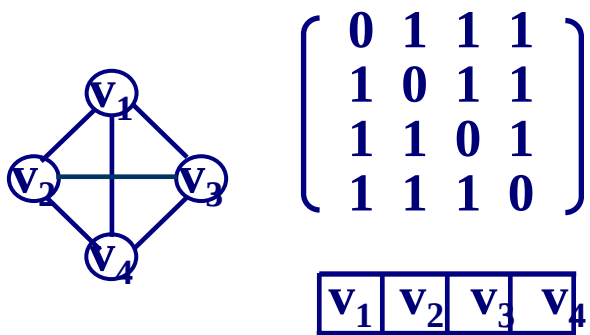
0 v_1
1 v_2
2 v_3
3 v_4

$$A_2 = \begin{bmatrix} \infty & 4 & \infty & \infty \\ 6 & \infty & 7 & \infty \\ 7 & \infty & \infty & \infty \\ \infty & \infty & 8 & \infty \end{bmatrix}$$

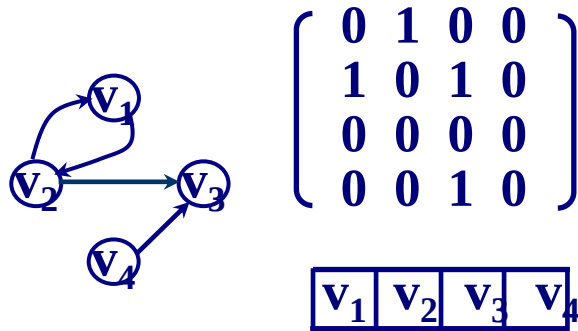


连接矩阵存储特点

- ◆ (1) 无向图的邻接矩阵一定是一个**对称矩阵**
- ◆ (2) 不带权的有向图的邻接矩阵一般是**稀疏矩阵**
- ◆ (3) 无向图的邻接矩阵的第i 行(或第i 列)非0 或非 ∞ 元素的个数为第i 个顶点的**度数**
- ◆ (4) 有向图的邻接矩阵的第i 行非0或非 ∞ 元素的个数为第i 个顶点的**出度**；第i 列非0或非 ∞ 元素的个数为第i 个顶点的**入度**



(无向图)



(有向图)

空间复杂度 $O(n^2)$



二. 邻接表存储方法

◆核心思想：建立n个线性链表存储该图

1. 每一个链表前面设置一个头结点,用来存放一个顶点的数据信息,称之为**顶点结点**。其构造为

vertex	link
--------	------

其中,vertex 域存放某个顶点的数据信息;

link 域存放某个链表中第一个结点的地址。

n个头结点之间为一数组结构。

2. 第i个链表中的每一个链结点(称之为**边结点**)表示以第i个顶点为**出发点**的一条边;边结点的构造为

adjvex	weight	next
--------	--------	------

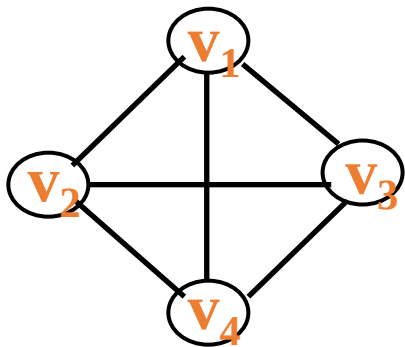
其中,next 域为指针域;

weight 域为权值域(若图不带权,则无此域);

adjvex 域存放以第i个顶点为出发点的一条边的另一端点在头结点数组中位置



例

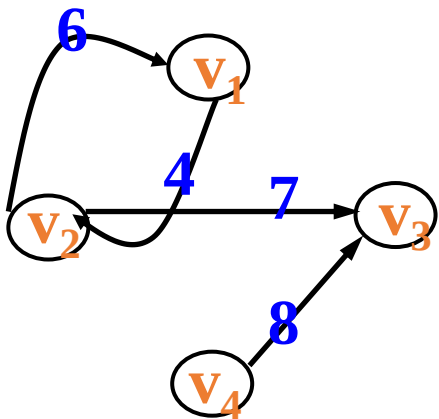


顶点结点

0	V ₁	-
1	V ₂	-
2	V ₃	-
3	V ₄	-

边结点

1	-	2	-	3	Λ
0	-	2	-	3	Λ
0	-	1	-	3	Λ
0	-	1	-	2	Λ



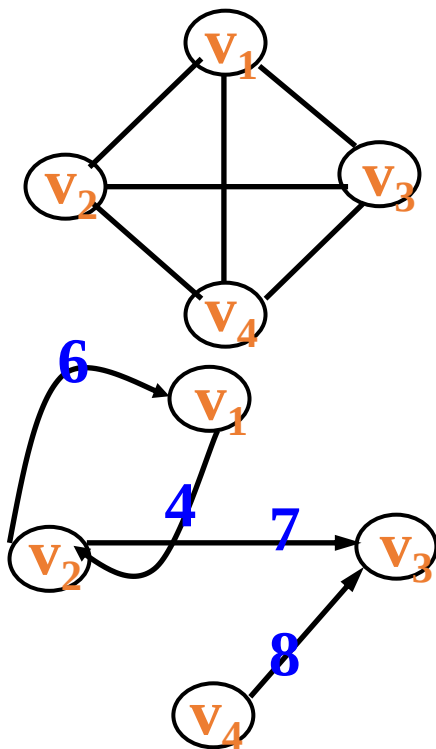
0	V ₁		→	1	4	Λ	
1	V ₂		→	0	6	—	→ 2 7 Λ
2	V ₃	Λ					
3	V ₄		→	2	8	Λ	



邻接表的特点

◆邻接表存储的特点

- ✓ (1) 无向图的第*i*个链表中边结点个数是第*i*个顶点度数
- ✓ (2) 有向图的第*i*个链表中边结点个数是第*i*个顶点的出度
- ✓ (3) 无向图的边结点个数一定为偶数；边结点个数为奇数的图一定是有向图

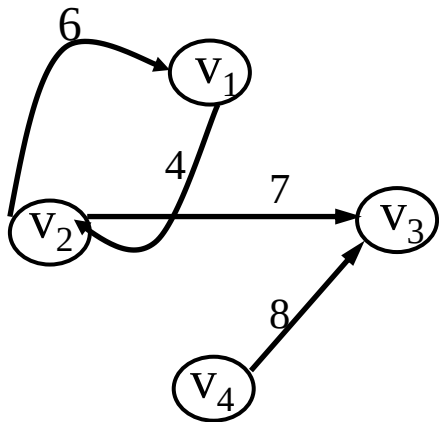


0	V ₁	→	1	→	2	→	3	Λ
1	V ₂	→	0	→	2	→	3	Λ
2	V ₃	→	0	→	1	→	3	Λ
3	V ₄	→	0	→	1	→	2	Λ

0	V ₁		→	1	4	Λ	
1	V ₂		→	0	6	→	2 7 Λ
2	V ₃	Λ					
3	V ₄		→	2	8	Λ	



逆邻接表



0	V ₁		→	1	4	∧	
1	V ₂		→	0	6	—	→ 2 7 ∧
2	V ₃	∧					
3	V ₄		→	2	8	∧	

邻接表

0	V ₁		→	1	6	∧	
1	V ₂		→	0	4	∧	
2	V ₃		→	1	7	-	→ 3 8 ∧
3	V ₄	∧					

逆邻接表

对于邻接表

第*i*个链表中的每一个链结点(称之为边结点)表示以第*i*个顶点为**出发点**的一条边; ...

终止点

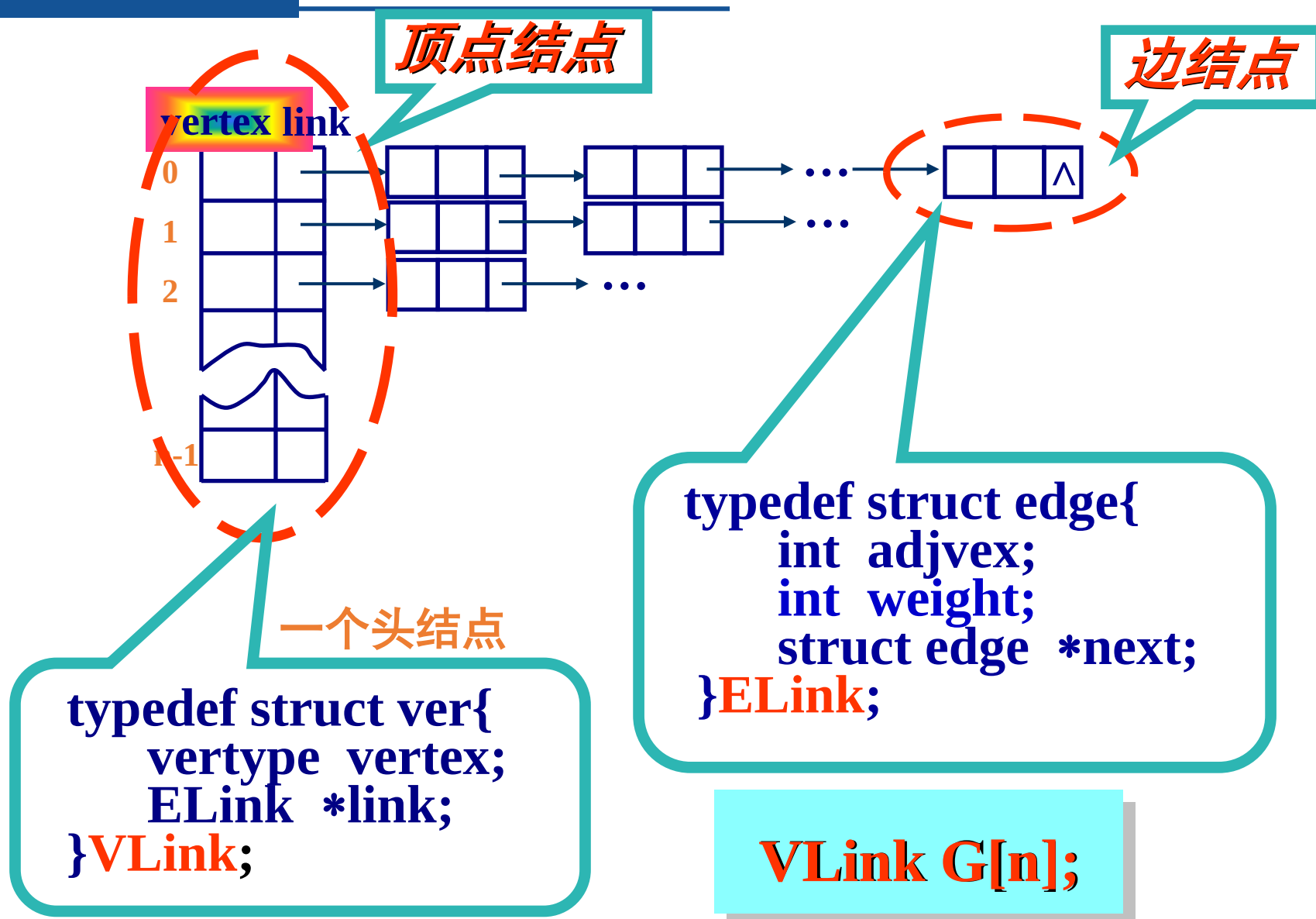


其他存储方法*

- ◆ 有向图的十字链表存储方法
- ◆ 无向图的多重邻接表存储方法



C语言实现：邻接表





C语言实现：邻接表（续）

#define MaxV <最大顶点个数>

```
typedef struct edge{  
    int adjvex;  
    int weight;  
    struct edge *next;  
}ELink;
```

定义边结点类型

```
typedef struct ver{  
    vertype vertex;  
    ELink *link;  
}VLink;
```

定义顶点结点类型

VLink G[MaxV];

邻接表



C语言实现：邻接矩阵

#define MaxV <最大顶点个数>

定义边类型

```
typedef struct edge{  
    int weight;  
    ...  
}Edge;
```

Vertype Vertex[MaxV];

顶点信息数组

Edge G[MaxV][MaxV];

邻接矩阵

邻接矩阵



C语言实现：边集数组（稀疏图）

```
#define MaxV <最大顶点个数>  
#define MaxE <最大边数>
```

```
typedef struct edge{  
    int v1, v2;  
    int weight;  
}Edge;
```

定义边类型

```
Vertype Vertex[MaxV];  
Edge G[MaxE];
```

顶点信息数组

边集数组



图的基本操作

```
createGraph();    // 创建一个图
destoryGraph();   // 删除一个图
insertVex(v);     // 在图中插入一个顶点v
deleteVex(v);     // 在图中删除一个顶点v
insertEdge(v, w); // 在图中插入一条边<v,w>
deleteEdge(v, w); // 在图中删除一条边<v,w>
traverseGraph();  // 遍历一个图
```



示例：创建一个图（邻接表）

若有如下输入：

```
8
0 2 4 ... -1
1 3 6 8 ... -1
...
```

第一行为图的顶点个数，从第二行开始第一个数为顶点序号，第二个数字开始为该顶点的邻接顶点，每行以-1结束，则创建一个邻接表存储的图算法如下：

```
void createGraph(VLink graph[])
{
    int i,n,v1,v2;
    scanf("%d",&n);
    for(i=0; i<n; i++){
        scanf("%d %d",&v1,&v2);
        while(v2 != -1){
            graph[v1].link=insertEdge(graph[v1].link, v2);
            graph[v2].link=insertEdge(graph[v2].link, v1);
            scanf("%d",&v2);
        }
    }
}
```

邻接矩阵：

$graph[v1][v2] = graph[v2][v1] = 1;$

```
#define MaxV 256
typedef struct edge{
    int adj;
    int wei;
    struct edge *next;
}Elink;
typedef struct ver{
    ELink *link;
}Vlink;
VLink G[MaxV];
```

//在链表尾插入一个节点

```
Elink *insertEdge(ELink *head, int avex)
{
    ELink *e,*p;
    e=(ELink *)malloc(sizeof(ELink));
    e->adj= avex; e->wei=1; e->next=NULL;
    if(head == NULL){head=e; return head;}
    for(p=head;p->next != NULL; p=p->next)
        ;
    p->next = e;
    return head;
}
```



提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题

7.3 图的遍历



某游客首次来京，想一次将旅游图上标明的景点玩到，他该如何设计线路即能将所有景点玩到，又不重复？



图的遍历

以无向图为例

从图中某个指定的顶点出发, 按照某一原则对图中所有顶点都访问一次且只访问一次, 得到一个由图中所有顶点组成的序列, 这一过程称为 **图的遍历**

利用图的遍历

- 确定图中满足条件的顶点;
- 求解图的连通性问题, 如求分量;
- 判断图中是否存在回路;
- ...



问题：独立路径问题（非简单图）

老张和老王酷爱爬山，每周必爬一次香山。有次两人为从东门到香炉峰共有多少条路径发生争执，于是约定一段时间内谁走过对方没有走过的路线多谁胜。

给定一线路图（无向连通图，两顶点之间可能有多条边），编程计算从起始点至终点共有多少条独立路径，并输出相关路径信息。

注：独立路径指的是从起点至终点的一条路径中至少有一条边是与别的路径中所不同的，同时路径中不存在环路。





一. 深度优先遍历(Depth First Search,DFS)

完成一个连通分量的遍历

类似于二叉树的前序遍历

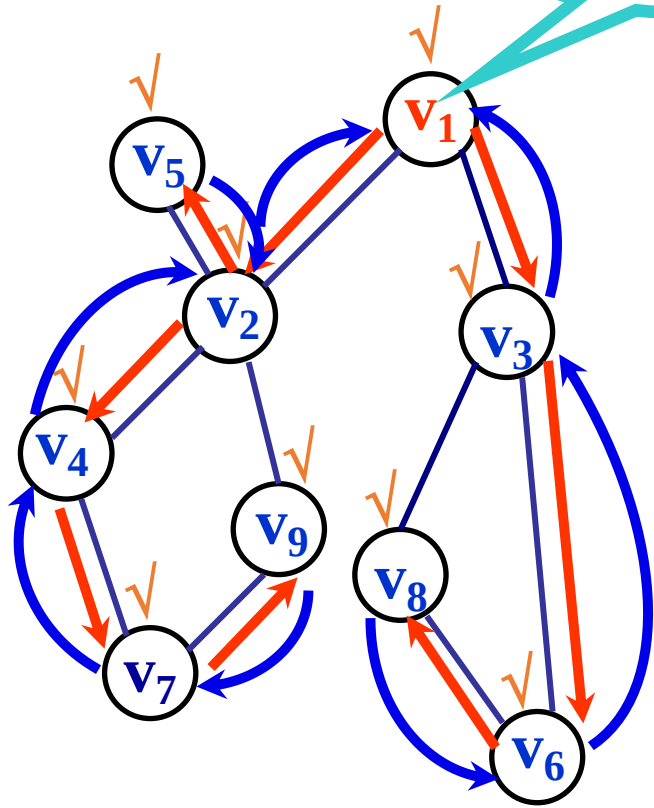
原则 从图中某个指定的顶点 v 出发,先访问顶点 v ,然后从顶点 v 未被访问过的一个邻接点出发,继续进行深度优先遍历,直到图中与 v 相通的所有顶点都被访问;★若此时图中还有未被访问过的顶点,则从另一个未被访问过的顶点出发重复上述过程,直到遍历全图。

递归过程



示例：深度优先遍历

出发点



关于遍历序列的唯一性

0	V ₁	-	1	-	2	∧
1	V ₂	-	0	-	3	-
2	V ₃	-	0	-	5	-
3	V ₄	-	1	-	6	∧
4	V ₅	-	1	∧		
5	V ₆	-	2	-	7	∧
6	V ₇	-	3	-	8	∧
7	V ₈	-	2	-	5	∧
8	V ₉	-	1	-	6	∧

1

2

3

4

8

∧

5

7

∧

V₁ V₂ V₄ V₇ V₉ V₅ V₃ V₆ V₈

遍历序列



如何确定顶点不被重复访问

为了标记某一时刻图中哪些顶点是否被访问,定义一维数组 $visited[0..n-1]$, 有

$$visited[i] = \begin{cases} 1 & \text{表示对应的顶点已经被访问} \\ 0 & \text{表示对应的顶点还未被访问} \end{cases}$$

算法分析

采用邻接表存储该图: $O(n+e)$

采用邻接矩阵存储该图: $O(n^2)$



算法实现

```
int Visited[N] = {0}; //标识顶点是否被访问过, N为顶点数
void travelDFS(VLink G[], int n){
    int i;
    for (i = 0; i < n; i++)
        Visited[i] = 0;
    for (i = 0; i < n; i++)
        if (!Visited[i])
            DFS(G, i);
}
```

```
void DFS(VLink G[], int v)
{
    ELink *p;
    Visited[v] = 1; //标识某顶点被访问过
    VISIT(G, v);    //访问某顶点
    for (p = G[v].link; p != NULL; p = p->next)
        if (!Visited[p->adjvex])
            DFS(G, p->adjvex);
}
```

//对比树的深度优先遍历算法

```
void DFStree(TNodeptr t){
    int i;
    if (t != NULL){
        VISIT(t); //访问t指向结点
        for (i = 0; i < MAXD; i++)
            if (t->next[i] != NULL)
                DFStree(t->next[i]);
    }
}
```





算法分析

- ◆如果图中具有 n 个顶点、 e 条边，则
 - ✓若采用邻接表存储该图,由于邻接表中有 $2e$ 个或 e 个边结点，因而扫描边结点的时间为 $O(e)$ ；而所有顶点都递归访问一次，所以，算法的时间复杂度为 $O(n+e)$
 - ✓若采用邻接矩阵存储该图，则查找每一个顶点所依附的所有边的时间复杂度为 $O(n)$ ，因而算法的时间复杂度为 $O(n^2)$



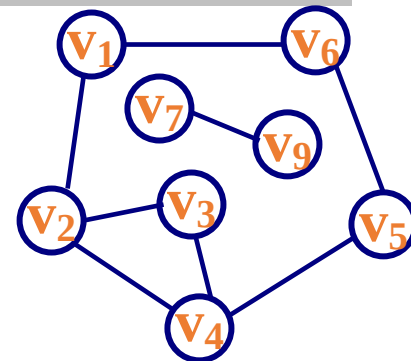
二.广度优先遍历(Breadth First Search,BFS)

类似于树的
按层次遍历

原则 从图中某个指定的顶点 v 出发,先访问顶点 v ,然后依次访问顶点 v 的各个未被访问过的邻接点,然后又从这些邻接点出发,按照同样的规则访问它们的那些未被访问过的邻接点,如此下去,直到图中与 v 相通的所有顶点都被访问;★若此时图中还有未被访问过的顶点,则从另一个未被访问过的顶点出发重复上述过程,直到遍历全图。★

完成所有连通
分量的遍历

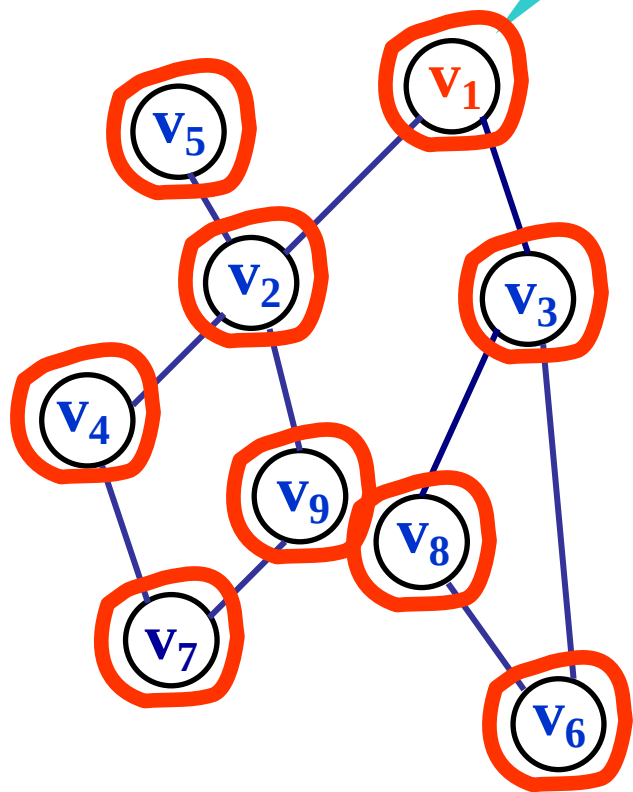
完成一个连通
分量的遍历





示例

出发点



0	V ₁	→	1	→	2	Λ
1	V ₂	→	0	→	3	→ 4 → 8
2	V ₃	→	0	→	5	→ 7
3	V ₄	→	1	→	6	Λ
4	V ₅	→	1	Λ		
5	V ₆	→	2	→	7	Λ
6	V ₇	→	3	→	8	Λ
7	V ₈	→	2	→	5	Λ
8	V ₉	→	1	→	6	Λ

队列

V₁ V₂ V₃ V₄ V₅ V₉ V₆ V₈ V₇

遍历序列



算法实现

```
int Visited[N] = {0}; //标识顶点是否被访问, N为顶点数
void travelBFS(VLink G[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        Visited[i] = 0;
    for (i = 0; i < n; i++)
        if (!Visited[i])
            BFS(G, i);
}
```

```
void BFS(VLink G[], int v){
    ELink *p;
    VISIT(G, v); //访问当前顶点
    Visited[v] = 1; //标识某顶点被访问过
    enqueue(Q, v);
    while (!emptyQ(Q)) {
        v = dequeue(Q); //取出队头元素
        p = G[v].link; //获取该顶点第一个邻接顶点
        //访问该顶点下的每个邻接顶点
        for (; p != NULL; p = p->next)
            if (!Visited[p->adjvex]) {
                VISIT(G, p->adjvex); //访问当前顶点
                Visited[p->adjvex] = 1; //标识某顶点被访问过
                enqueue(Q, p->adjvex);
            }
    }
}
```



DFS与BFS

◆对比这两个图的遍历算法

- ✓它们在时间复杂度上是一样的
- ✓不同之处仅仅在于对顶点的访问的顺序不同

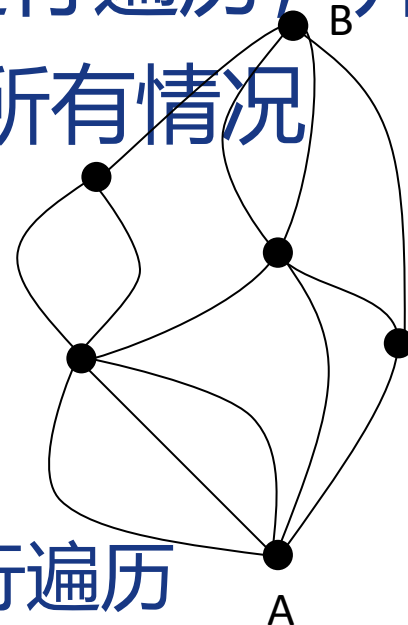
◆具体用哪个取决于具体问题

- ✓通常DFS更适合目标比较明确，以找目标为主要目的的情况
- ✓BFS更适合在不断扩大遍历范围时找到相对最优解的情况



问题：独立路径计算 – 问题分析与算法设计

- ◆本问题的实质：给定起点（如图中A），对图进行遍历，并在遍历图的过程中找到到达终点（如图中B）的所有情况
 - ✓DFS和BFS算法都是从源点出发对邻接顶点的遍历
 - ✓而问题是本文中两个点间可能有多个边（如图所示）
- ◆算法策略是对DFS算法（或BFS）进行改进
 - ✓原来按邻接顶点进行遍历，改为按邻接顶点的边进行遍历
 - ✓即从一个顶点出发遍历其邻接顶点时，按邻接顶点的边进行深度遍历，即只有当某顶点的所有邻接顶点的所有边都遍历完才结束该结点的遍历





问题：独立路径计算 – 数据结构设计

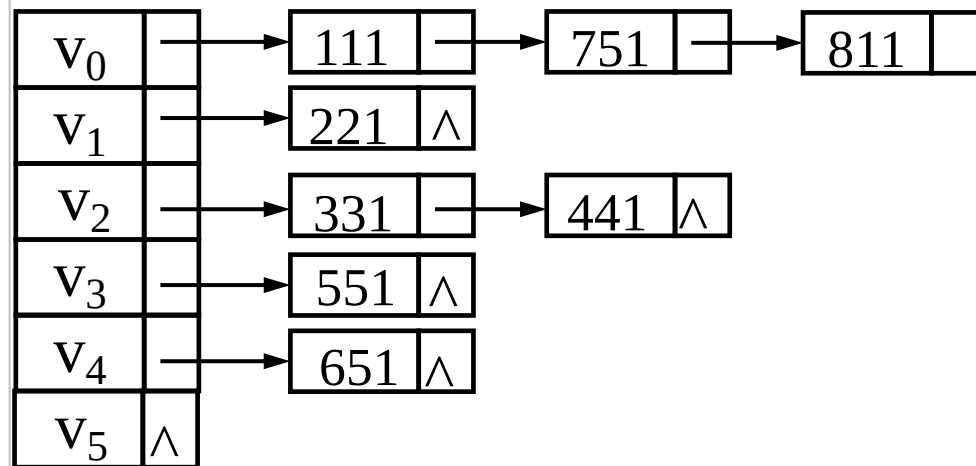
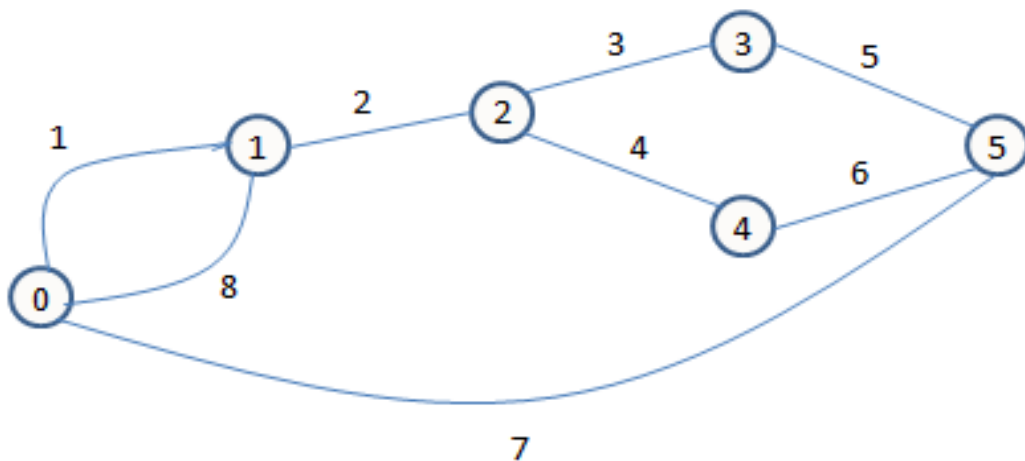
采用邻接表来存储图，邻接表设计如下：

```
#define MAXSIZE 512
struct edge{//边结点结构
    int eno;    //边序号
    int adjvex; //邻接顶点
    //边的权重，可为距离或时间，本例为1
    int weight;
    struct edge *next;
};
```

```
struct ver
{ //顶点结构，邻接表下标即为顶点序号
    struct edge *link;
};
struct ver G[MAXSIZE]; //由邻接表构成的图
char Visted[MAXSIZE]={0}; //标识相应顶点是否被访问
int paths[MAXSIZE]; //独立路径
```

【样例输入】

```
6 8
1 0 1
2 1 2
3 2 3
4 2 4
5 3 5
6 4 5
7 0 5
8 0 1
```





问题：独立路径计算 – 主要代码*

//基于DFS的独立路径查找算法

```
void eDFS(int v, int level){
    struct edge *p;
    if(v == V1) {printPath(level); return;}
    for(p=Graph[v].link; p!= NULL; p=p->next)
        if( !Visited[p->adjvex]){
            Paths[level] = p->eno;
            Visited[p->adjvex] = 1;
            eDFS(p->adjvex, level+1);
            Visited[p->adjvex] = 0;
        }
}
```

//原来的DFS算法

```
void DFS(VLink G[ ], int v) {
    ELink *p;
    Visited[v] = 1; //标识某顶点被访问过
    VISIT(G, v); //访问某顶点
    for(p = G[v].link; p !=NULL; p=p->next )
        if( !Visited[p->adjvex] )
            DFS(G, p->adjvex);
}
```

```
int V0, V1;
int main(){
    int en,eno,i,v1,v2;
    scanf("%d %d", &Vnum, &en);
    for(i=0; i<en; i++){
        scanf("%d %d %d", &eno, &v1, &v2);
        Graph[v1].link = insertEdge(Graph[v1].link,v2, eno);
        Graph[v2].link = insertEdge(Graph[v2].link,v1, eno);
    }
    V0 = 0; V1 = Vnum-1; Visited[V0] = 1;
    eDFS(V0,0);
    return 0;
}
```

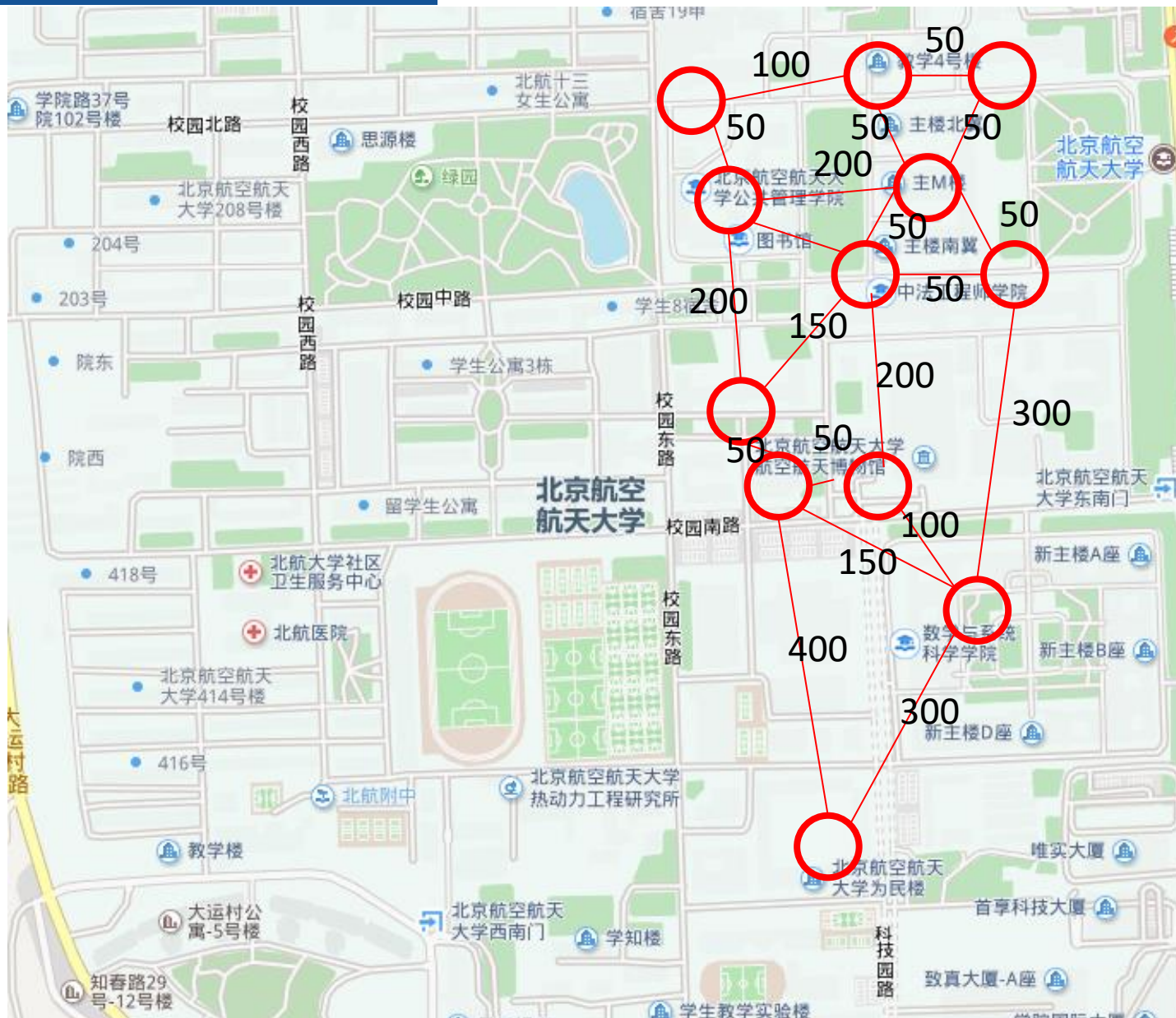


提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题



7.4 最小生成树(Minimum Cost Spanning Tree)

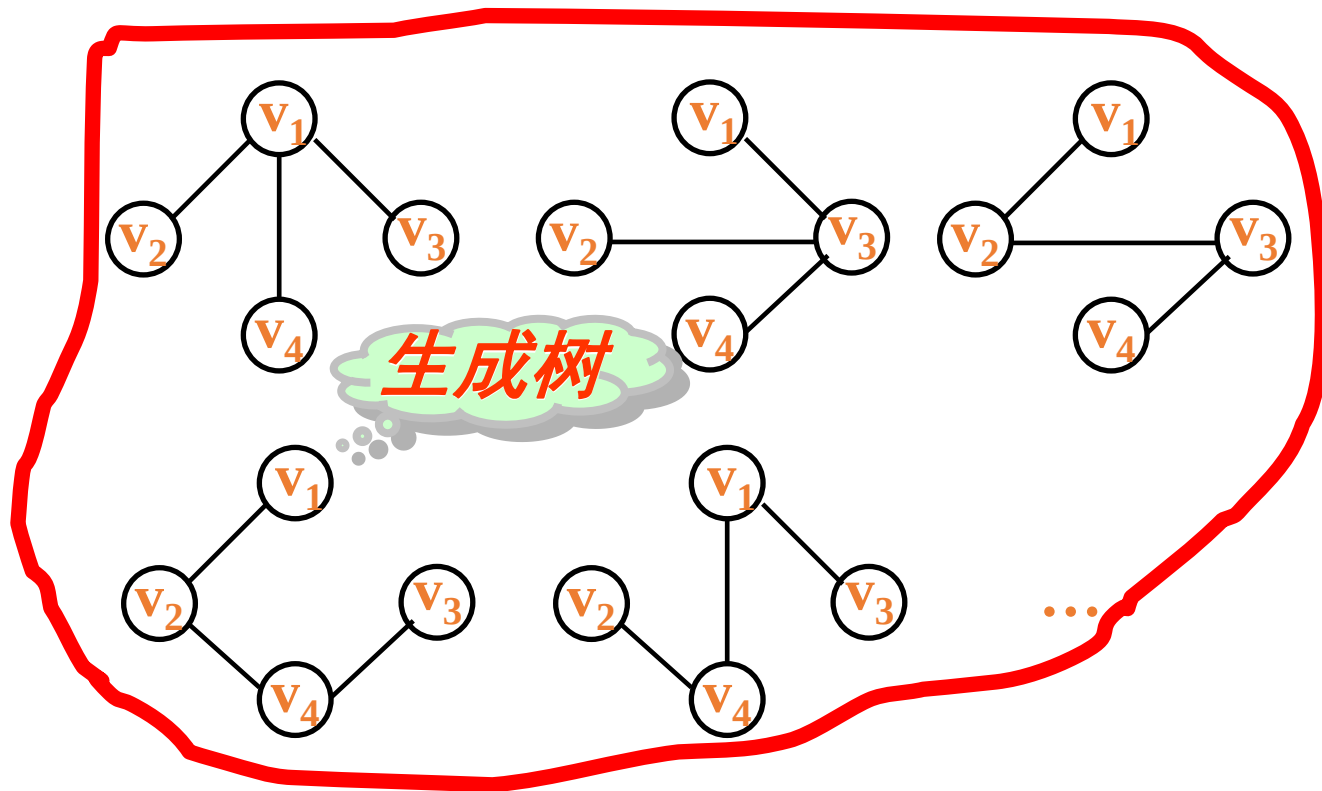
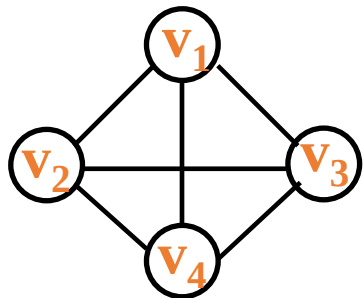


问题：北航网络中心要给北航主要办公楼间铺设光缆以构建网络。如何以最小的成本完成网络铺设？



生成树

◆生成树：包含着连通图的全部 n 个顶点，仅包含其 $n-1$ 条边的极小连通子图





生成树的性质

◆生成树：包含 n 个顶点的连通图 G 的全部 n 个顶点，仅包含其 $n-1$ 条边的极小连通子图，称为 G 的一个生成树

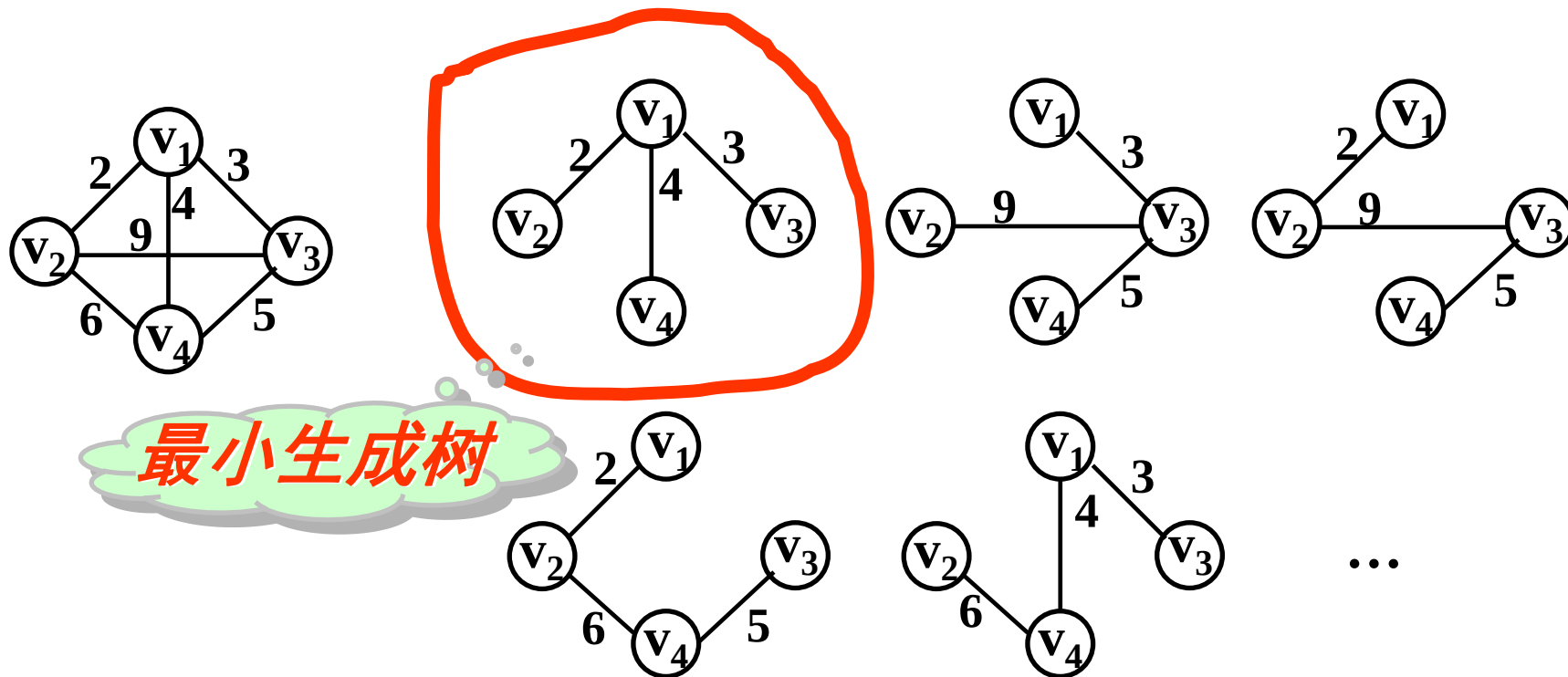
◆性质

1. 包含 n 个顶点的图：连通且有 $n-1$ 条边
 - 无回路且有 $n-1$ 条边
 - 无回路且连通
 - 是一棵树
2. 如果 n 个顶点的图中只有少于 $n-1$ 条边，图将不连通
3. 如果 n 个顶点的图中有多于 $n-1$ 条边，图将有环（回路）
4. 一般情况下，生成树不唯一



最小生成树

- ◆带权连通图中，**总权值之和最小**的带权生成树为最小生成树
 - ✓最小生成树也称最小代价生成树,或最小花费生成树





构造最小生成树的原则

- ◆只能利用图中的边来构造最小生成树
- ◆只能使用、且仅能使用图中的 $n-1$ 条边来连接图中的 n 个点
- ◆不能使用图中产生回路的边



二. 求最小生成树：普里姆(Prim)算法

普里姆(Prim)算法

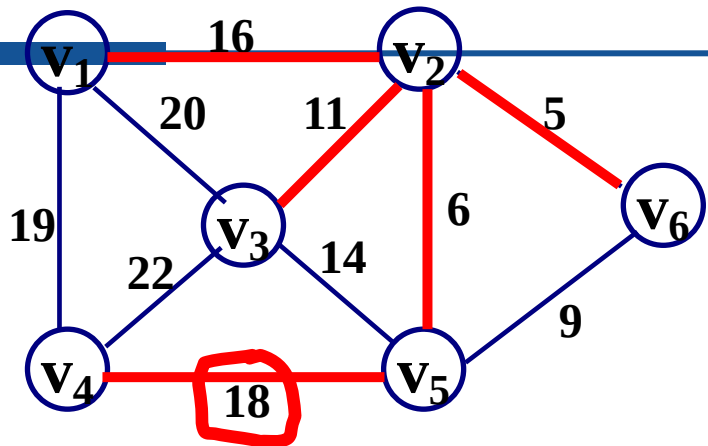
设 $G=(V, GE)$ 为具有 n 个顶点的带权连通图;
 $T=(U, TE)$ 为生成的最小生成树,
初始时, $TE=空$, $U=\{v\}$, $v \in V$ 。

依次在 G 中选择一条一个顶点仅在 V 中,另一个顶点在 U 中,并且权值最小的边加入集合 TE , 同时将该边仅在 V 中的那个顶点加入集合 U 。

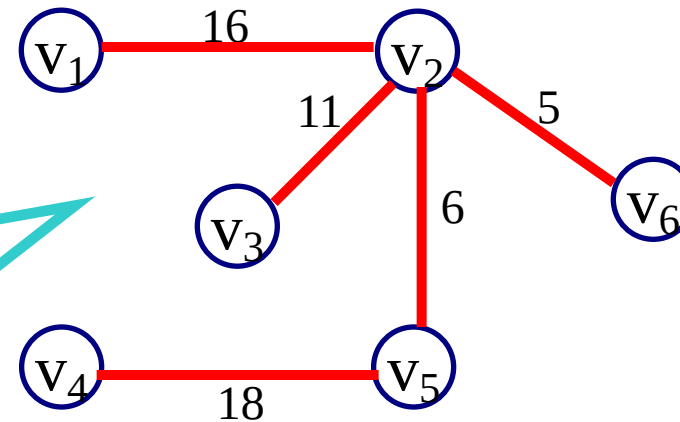
重复上述过程 $n-1$ 次,使得 $U=V$,此时 T 为 G 的最小生成树。



示例



最小生成树



最小生成树的权值 = 56

G:

$$V = \{ v_1, v_2, v_3, v_4, v_5, v_6 \}$$

$$GE = \left\{ \begin{array}{ll} (v_1, v_2)_{16}, & (v_1, v_3)_{20} \\ (v_1, v_4)_{19}, & (v_2, v_3)_{11} \\ (v_2, v_5)_{6}, & (v_2, v_6)_{5} \\ (v_3, v_4)_{22}, & (v_3, v_5)_{14} \\ (v_4, v_5)_{18}, & (v_5, v_6)_{9} \end{array} \right\}$$

T:

$$U = \{ v_1, v_2, v_6, v_5, v_3, v_4 \}$$

$$TE = \left\{ \begin{array}{ll} (v_1, v_2)_{16} & (v_2, v_6)_{5} \\ (v_2, v_5)_{6} & (v_2, v_3)_{11} \\ (v_4, v_5)_{18} & \end{array} \right\}$$

最小生成树唯一吗?



Prim算法主要的数据结构

`int weights[MAXVER][MAXVER];`

当图G中存在边(i,j), 则weights[i][j]为其权值, 否则为一个INFINITY

`int edges[MAXVER];`

存入生成的最小生成树的边, 如: (i, edges[i]) 为最小生成树的一条边, 应有n-1条边

`int minweight[MAXVER];`

存放未确定为生成树的顶点至已确定的生成树上顶点的边权重, minweight[i] = 0 表示其已确定为最小生成树顶点



算

```
#define MAXVER 512
#define INFINITY 32767
void Prim(int weights[][MAXVER], int n, int src, int edges[])
{ // weights为权重数组、n为顶点个数、src为最小树的第一个顶点、edge为最小生成树边
    int minweight[MAXVER], min;
    int i, j, k;
    for (i = 0; i < n; i++){ //初始化相关数组
        minweight[i] = weights[src][i]; //将src顶点与之有边的权值存入数组
        edges[i] = src; //初始化第一个顶点为src
    }
    minweight[src] = 0; //将第一个顶点src顶点加入生成树
    for (i = 1; i < n; i++) { min = INFINITY;
        for (j = 0, k = 0; j < n; j++)
            if (minweight[j] != 0 && minweight[j] < min) { //在数组中找最小值，其下标为k
                min = minweight[j]; k = j;
            }
        minweight[k] = 0; //找到最小树的一个顶点
        for (j = 0; j < n; j++)
            if (minweight[j] != 0 && weights[k][j] < minweight[j]) {
                minweight[j] = weights[k][j]; //将小于当前权值的边(k,j)权值加入数组中
                edges[j] = k; //将边(j,k)信息存入边数组中
            }
        }
    }
}
```

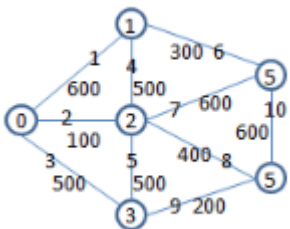


问题：北航网络中心铺设光缆



【样例输入】

```
6 10
1 0 1 600
2 0 2 100
3 0 3 500
4 1 2 500
5 2 3 500
6 1 4 300
7 2 4 600
8 2 5 400
9 3 5 200
10 4 5 600
```



设计考虑：

1. 可用邻接矩阵存储网络图

数据结构：

```
struct edge {
    int id;
    int wei;
};
```

```
struct edge graph[MAXVER][MAXVER]; // 邻接矩阵
int edges[MAXVER] = {0}; // 生成树数组
```

根据输入值对<id,v1,v2,wei>构造图：

```
graph[v1][v2].id = id; graph[v1][v2].weight = wei;
graph[v2][v1].id = id; graph[v2][v1].weight = wei;
```

- 调用Prim算法得到最小生成树，存放在edges数组中
- 根据生成树数组edges可得到生成树边序号 为 graph[i][edges[i]].id 的边，其权重为： graph[i][edges[i]].wei
- 最小生成树按边序号进行排序输出。



克鲁斯卡尔 (Kruskal) 算法

基本思想

$G=(V, GE)$
 $T=(U, TE)$
初始时, $TE=\text{空}$
 $U=V$

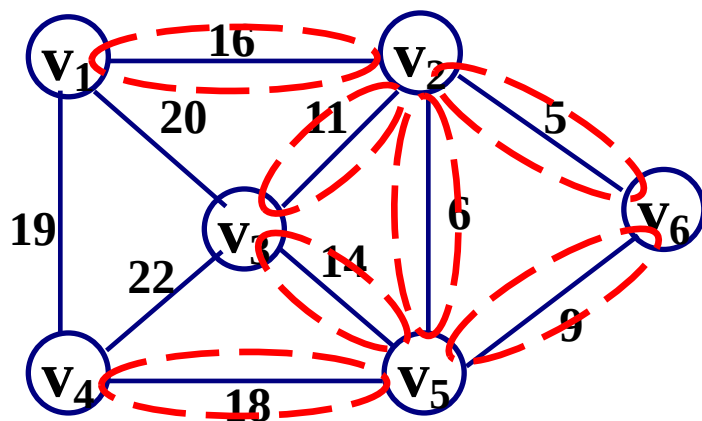
从G中选择一条**当前未选择过的、且边上的权值最小**的边加入TE, 若加入TE后使得T**未产生回路**, 则**本次选择有效**, 如使得T产生回路, 则本次选择无效, 放弃本次选择的边

重复上述选择过程直到TE中包含了G的 **$n-1$ 条边**, 此时的T为G的**最小生成树**

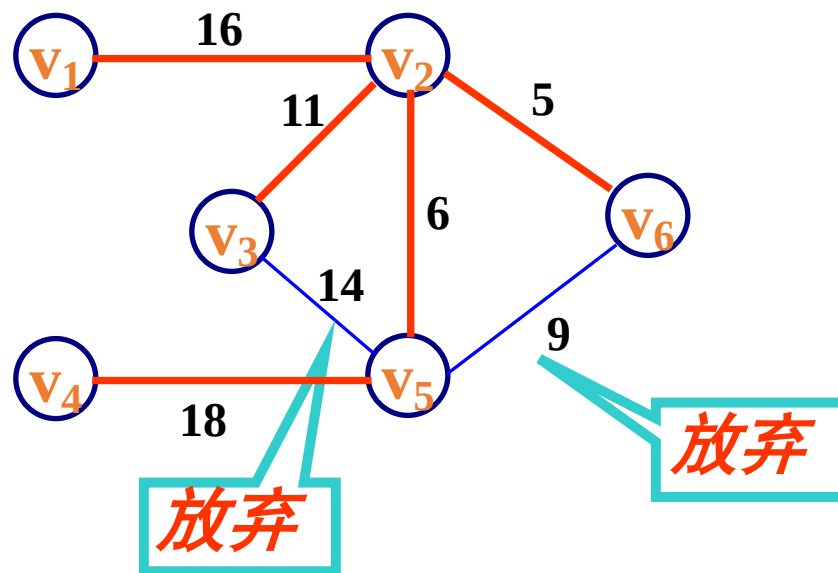


示例

G (连通图)



T (生成树)



克鲁斯卡尔(Kruskal)算法



关于最小生成树的一些特点

- ◆✓任意连通图中，假设没有相同权值的边存在，则权值最小的边一定是其最小生成树中的边
- ◆✗任意连通图中，假设没有相同权值的边存在，则权值最大的边一定不是其最小生成树中的边
- ◆✓任意连通图中，假设没有相同权值的边存在，则与同一顶点相连的权值最小的边一定是其最小生成树中的边
- ◆✗采用克鲁斯卡尔算法求最小生成树的过程中，判断一条待加入的边是否形成回路，只需要判断该边的两个顶点是否都已经加入到集合U中
- ◆对比两个算法，Kruskal算法主要是针对边展开，边数少时效率会非常高，所以对稀疏图有很大的优势；Prim算法对于稠密图，即边数非常多的情况会好一些



提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题



问题7.1：北京地铁乘坐线路查询

◆编写一个程序实现北京地铁乘坐线路查询，输入为起始站名和终点站名，输出为从起始到终点的换乘线路。

◆要求给出：

✓乘坐站数最少的换乘方式（理论最快，通常用于计算票价），如从西土城到北京西站：

➢西土城-10-黄庄-4-国家图书馆-9-北京西站

✓换乘次数最少的换乘方式（最方便），如

➢西土城-10-六里桥-9-北京西站

最短路径问题





7.5 最短路径问题

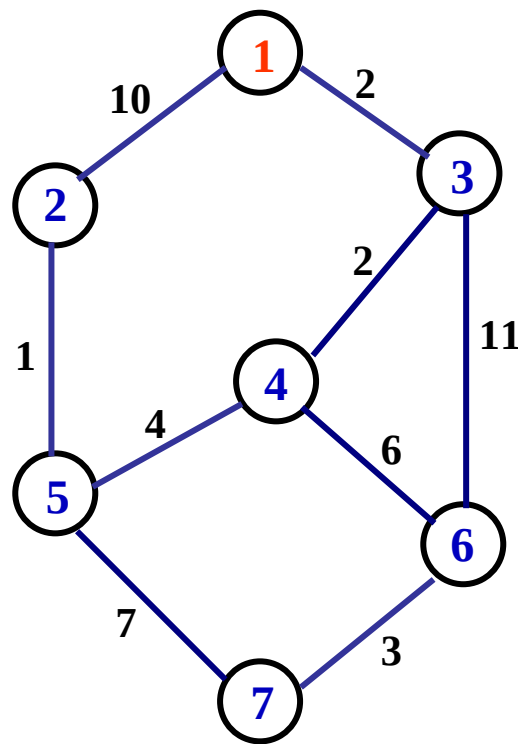
一、路径长度的定义

不带权的图： 路径上所经过的边的数目

带权的图： 路径上所经过的边的权值之和

二、单源最短路径问题

设出发顶点为 v (通常称为**源点**),
单源最短路径求从顶点 v 到达其它节点的
最短路径即为**单源最短路径**





单源最短路径的迪杰斯特拉(Dijkstra)算法

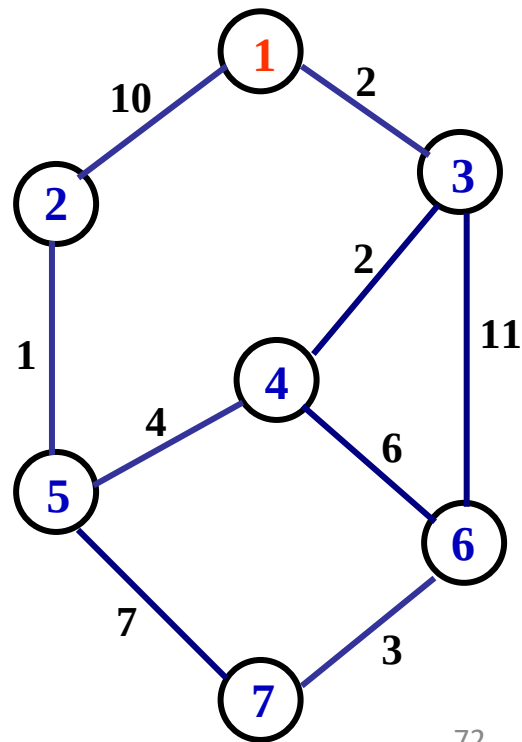
1. 基本思想

(1) 设 $G=(V,E)$ 是一个带权有/无向图, 把图中顶点集合 V 分成两组:

- 第一组为**已求出最短路径的顶点集合**(用 S 表示, 初始时 S 中只有一个源, 以后每求得一条最短路径 $v, \dots, v_k$, 就将 v_k 加入到集合 S 中, 直到全部顶点都加入到 S 中)
- 第二组为**其余未确定最短路径的顶点集合**(用 U 表示)

(2) 按**最短路径长度的递增次序**依次把第二组的顶点加入 S 中。在加入的过程中, 总保持从源点 v 到 S 中各顶点的最短路径长度**不大于**从源点 v 到 U 中任何顶点的最短路径长度
每个顶点对应一个距离:

- S 中顶点的距离: 从 v 到此顶点的最短路径长度
- U 中顶点的距离: 从 v 到此顶点只包括以 S 中的顶点为中间顶点的当前最短路径长度





Dijkstra算法 (续)

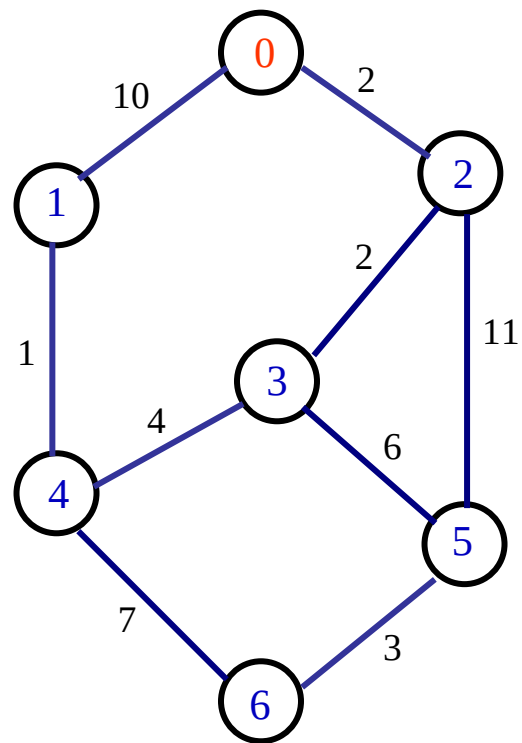
2. 图的存储

以1~n 分别代表n个顶点,采用邻接矩阵存储该图,有

$$A[i][j] = \begin{cases} W_{ij} & \text{当顶点} v_i \text{ 到顶点} v_j \text{ 有边,且权为} W_{ij} \\ \infty & \text{当顶点} v_i \text{ 到顶点} v_j \text{ 无边时} \\ 0 & \text{当} v_i = v_j \text{ 时} \end{cases}$$

0	10	2	∞	∞	∞	∞
10	0	∞	∞	1	∞	∞
2	∞	0	2	∞	11	∞
∞	∞	2	0	4	6	∞
∞	1	∞	4	0	∞	7
∞	∞	11	6	∞	0	3
∞	∞	∞	∞	7	3	0

邻接矩阵





Dijkstra算法 (续)

3. 从源点到当前各顶点的最短路径长度的存储

用数组 $Sweight[0..n-1]$ 表示, 一般情况下,

$Sweight[k] = \langle \text{源点到顶点 } k \text{ 的弧上的权值} \rangle$

或者 = $\langle \text{源点到 } S \text{ 中其它顶点的路径长度} \rangle$

+ $\langle S \text{ 中其它顶点到顶点 } k \text{ 的弧上的权值} \rangle$

初始时, $Sweight$ 数组的值为邻接矩阵
中顶点 v 对应的边值信息

0	10	2	∞	∞	∞	∞
10	0	∞	∞	1	∞	∞
2	∞	0	2	∞	11	∞
∞	∞	2	0	4	6	∞
∞	1	∞	4	0	∞	7
∞	∞	11	6	∞	0	3
∞	∞	∞	∞	7	3	0

邻接矩阵



Dijkstra算法 (续)

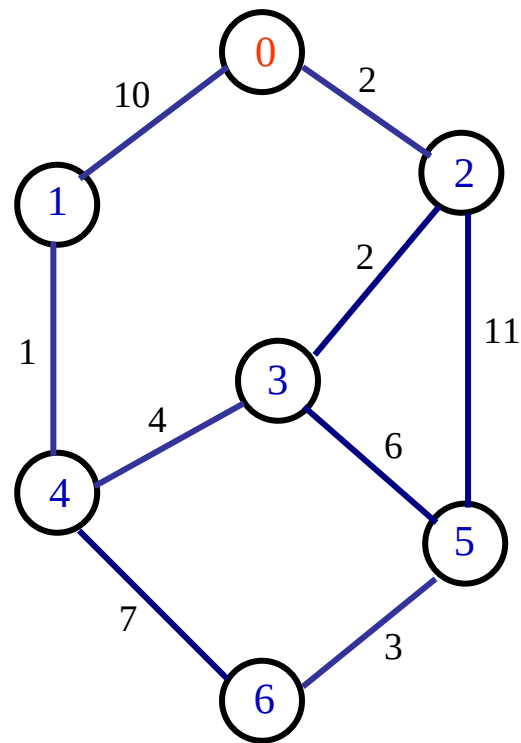
4.标识当前已找到最短路径的顶点

用数组wfound[0..n-1]表示, 有:

$$wfound[k] = \begin{cases} 1 & \text{已经找到最短路径} \\ 0 & \text{尚未找到最短路径} \end{cases}$$

初始时,

$$wfound[v]=1, wfound[k]=0 \quad (v=0, k=1, 2, \dots, n-1, \text{且} k \neq v)$$





Dijkstra算法 (续)

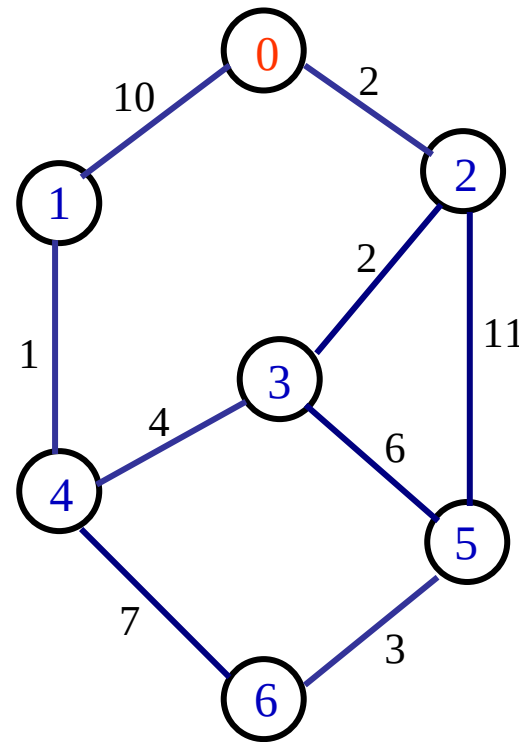
5.记录各顶点最短路径所经过的顶点序列

用数组 $Spath[0..n-1]$ 表示, 其中,

$Spath[k]$ 保存从源点到顶点 v_i 当前最短路径中的前一个顶点的编号, 初始时,

当源点 V 到 V_k 有边时, $Spath[k]=\{v\}$, $k=0, 1, \dots, n-1$

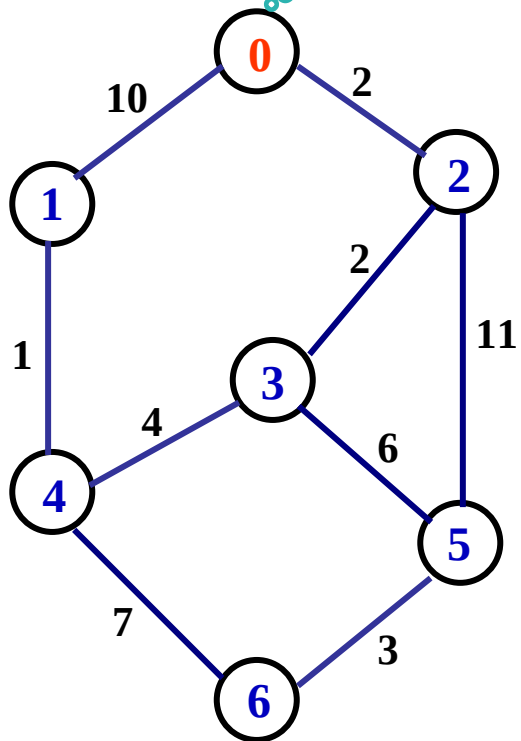
当源点 V 到 V_k 无边时, $Spath[k]=-1$





示例

源点 $v=0$



Sweight数组

0	10	2	∞	∞	∞	∞
---	----	---	----------	----------	----------	----------

wfound数组

1	0	0	0	0	0	0
---	---	---	---	---	---	---

Spath数组

0	0	0				
---	---	---	--	--	--	--

0	10	2	∞	∞	∞	∞
10	0	∞	∞	1	∞	∞
2	∞	0	2	∞	11	∞
∞	∞	2	0	4	6	∞
∞	1	∞	4	0	∞	7
∞	∞	11	6	∞	0	3
∞	∞	∞	∞	7	3	0

邻接矩阵

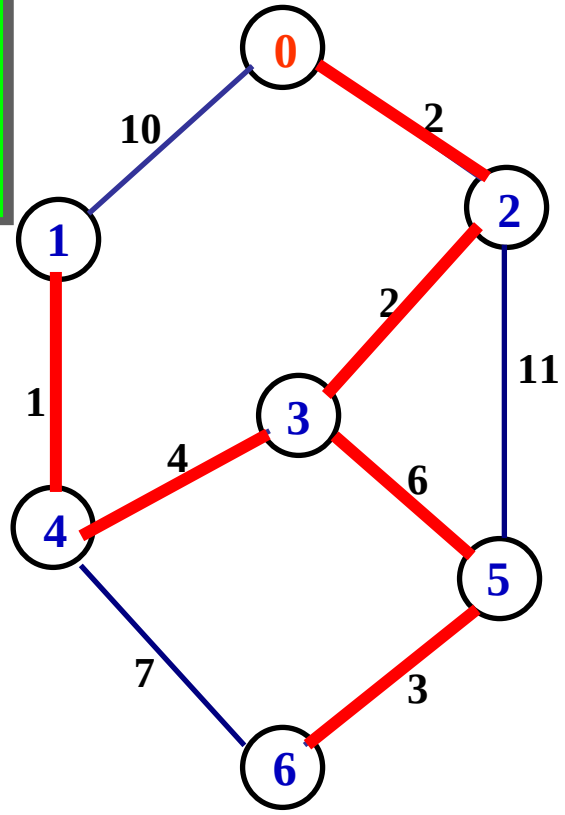


Dijkstra算法示例

顶点	0	1	2	3	4	5	6
Weight	0	10	2	∞	∞	∞	∞
	0	10	2	4	∞	13	∞
	0	10	2	4	8	10	∞
	0	9	2	4	8	10	15
	0	9	2	4	8	10	15
	0	9	2	4	8	10	13
	0	9	2	4	8	10	13
Wfound	1	1	1	1	1	1	1
Spath	0	4	0	2	3	3	5

路径长度

源点: $v=0$



最短路径最后经由点



Dijkstra算法（续）：算法描述

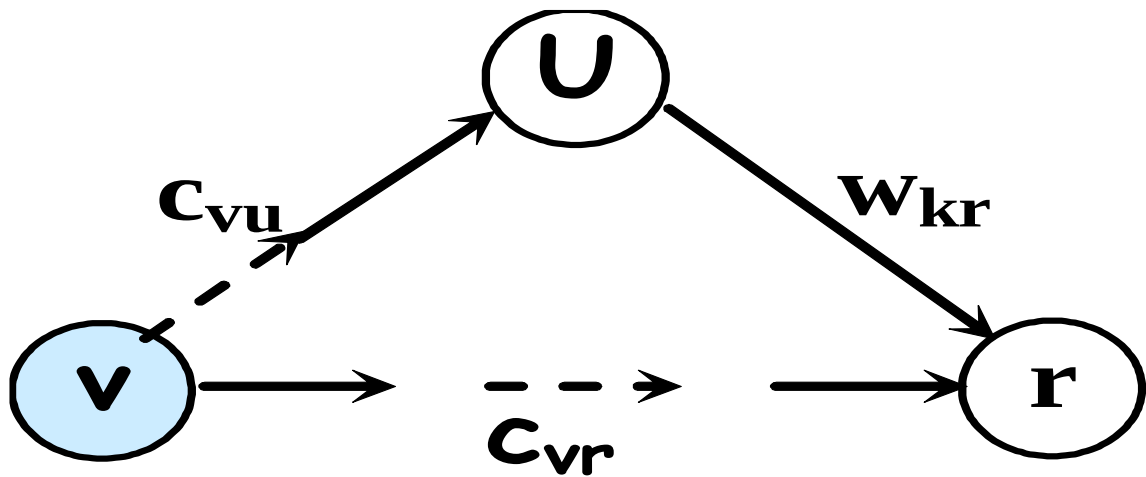
1. 确定Sweight、wfound、Spath 三个数组的初值
2. 利用wfound数组与Sweight 数组在那些尚未找到最短路径的顶点中确定一个与源点v最近的顶点u,并置wfound[u]为1
3. 根据顶点u修改源点到所有尚未找到最短路径的顶点的路径长度，并修改相应Spath值
4. 重复上述过程的第2至第3步 $n-1$ 次



Dijkstra算法（续）：算法描述

3.根据顶点u修改源点到所有尚未找到最短路径的顶点的路径长度，即

1)将源点v到顶点u(最短)路径长度分别加到源点v通过顶点u可以**直接到达、且尚未找到最短路径的那些顶点**的路径长度上



$$Sweight[r] = \text{MIN}(c_{vu} + w_{ur}, c_{vr})$$

2)若替换，则Spath[r]=u。



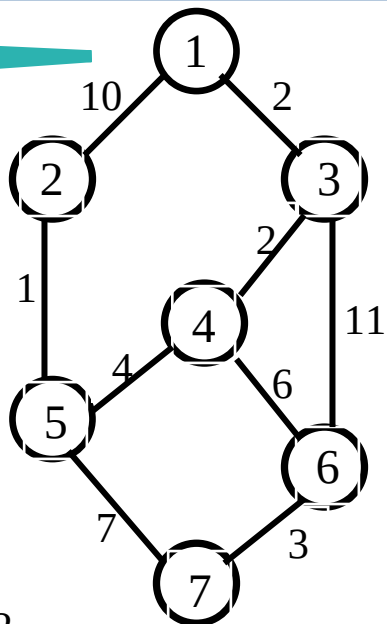
Dijkstra算法（续）：算法实现

```
Dijkstra(int v0){
    int i, j, v, minweight;
    char wfound[VNUM] = {0}; //用于标记从v0到相应顶点是否找到最短路径, 0未找到, 1找到
    for (i = 0; i < VNUM; i++) {
        Sweight[i] = Weights[v0][i];  Spath[i] = v0; } //初始化数组Sweight和Spath
    Sweight[v0] = 0;    wfound[v0] = 1;
    for (i = 0; i < VNUM - 1; i++) { //迭代VNUM-1次
        minweight = INFINITY;
        for (j = 0; j < VNUM; j++) //找到未标记的最小权重值顶点
            if (!wfound[j] && (Sweight[j] < minweight)) {
                v = j;  minweight = Sweight[v]; }
        wfound[v] = 1; //标记该顶点为已找到最短路径
        for(j=0; j<VNUM; j++)//找到未标记顶点且权值大于v权值+(v,j)的权值, 更新其权值
            if (!wfound[j] && (minweight + Weights[v][j] < Sweight[j])) {
                Sweight[j] = minweight + Weights[v][j];
                Spath[j] = v; //记录前驱顶点
            }
    }
}
```



思考：最小生成树与最短路径生成树

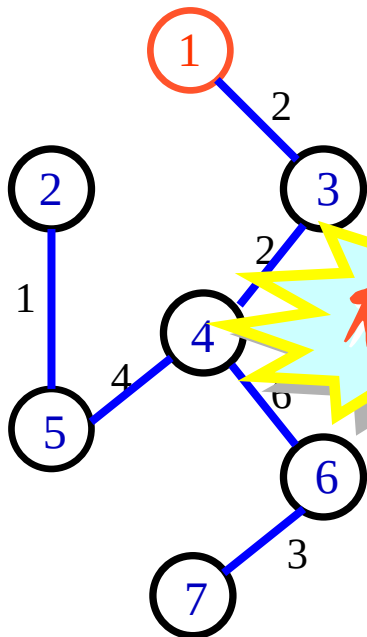
源点



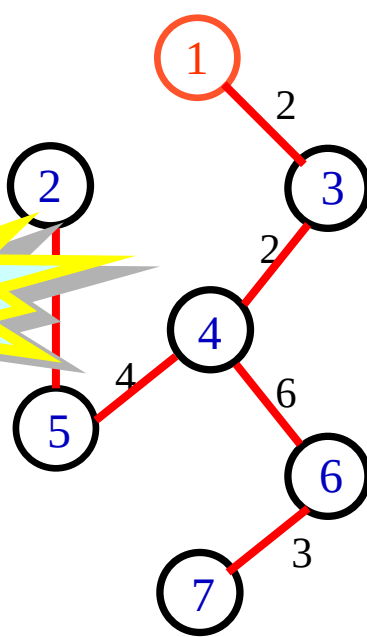
问题

对于给定的带权连通无向图，从某源点到图中各顶点的最短路径构成的生成树是否是该图的最小生成树？为什么？

最小代价生成树



相同



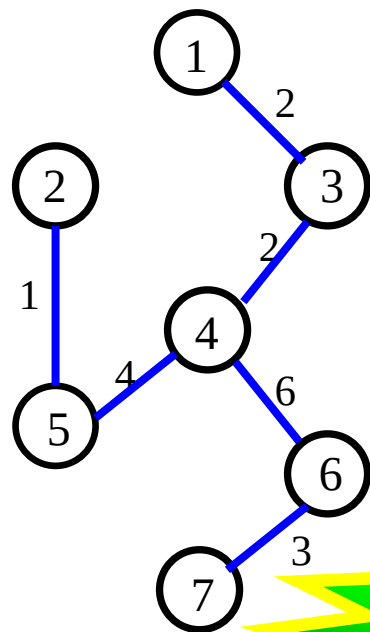
最短(路径)生成树



思考：最小生成树与最短路径生成树

与“源点”无关！

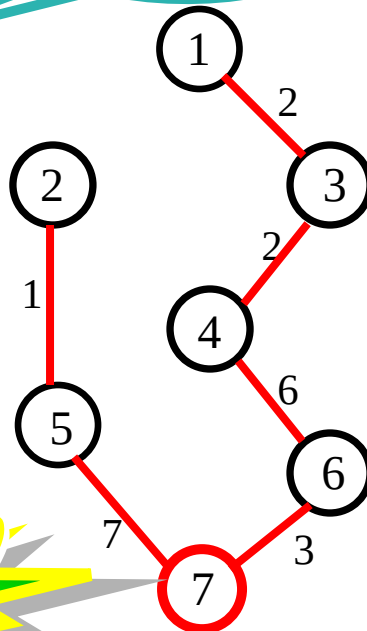
最小代价生成树



源点

与源点有关！

最短(路径)生成树

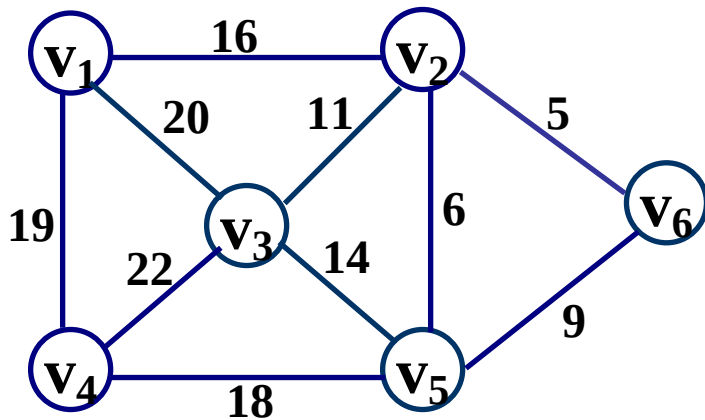


不相同！

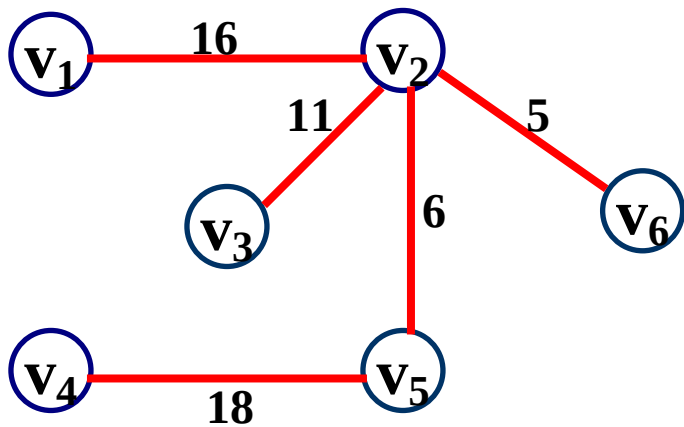


再看一个例子

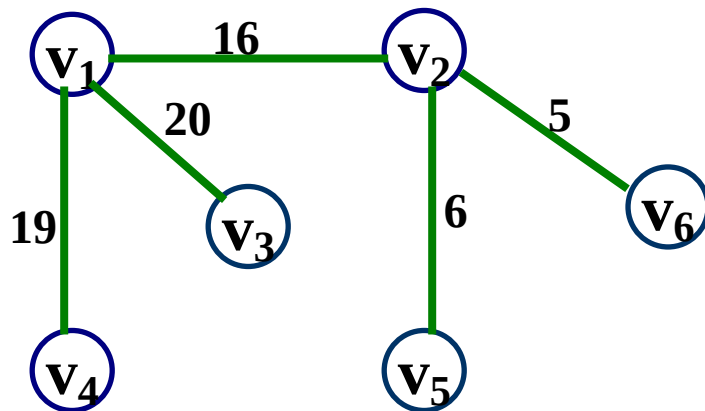
再看一个例子



结论...



最小生成树



源点为 v_1 的最短路径

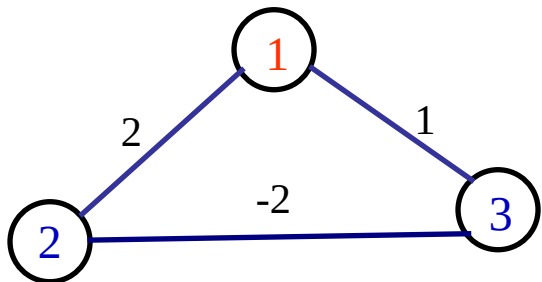


Dijkstra算法的局限性*

对于带有负权值的图?

◆考虑从1-3的最短路径:

- ✓1->3: 路径长度1, Dijkstra输出结果
- ✓1->2->3: 路径长度0, 更短!



◆问题所在?

- ✓Dijkstra算法要求已经确定的最短路径在后续算法执行时不会被修改。即路径长度要单调递增。
- ✓解决方法: re-weighting, 每条路径增加2? 仍然不解决问题!
- ✓如果**存在负回路**, 则最短路径可以任意负下去!
- ✓直接求s-t的简单最短路径是难解问题!
- ✓Bellman-Ford algorithm (无负回路)



更多的最短路径算法*

◆除Dijkstra算法，其他最短路径算法：

✓Floyd算法（Floyd-Warshall算法）

- 非常简洁，特别适用于计算所有顶点间的最短路径问题
- 算法复杂度为 $O(n^3)$

✓Bellman-Ford算法

- 从DJ算法引申出来的，可以解决带有负权边的最短路径问题

✓SPFA算法

- 可以解决负权边的问题，且复杂度比Ford低
- 可以看做是dj算法和BFS算法

✓A*算法



问题7.1：北京地铁乘坐线路查询

文件bgstations.txt为数据文件，包含了北京地铁的所有线路及所有车站信息。其格式如下：

12
1 23
苹果园0
古城0
...
公主坟1
...
四惠东1
...
2 19
西直门1
积水潭0
...
西直门

说明：表明目前北京地铁共开通12条线，其中1号线有23个车站，分别为苹果园，非换乘站；...；公主坟，换乘站...。2线共有19个站，分别为西直门，换乘站，...。

输入：

起始站：西土城

目的站：北京西站

输出：西土城-10(1)-知春路-13(2)-西直门-4(2)-国家图书馆-9(4)-北京西站





问题7.1：数据结构设计

```
#define MAXNUM 512    //地铁最大站数
#define MAXLEN 16     //地铁站名的最大长度
#define INFINITY 32767

struct station { //车站信息
    char sname[MAXLEN]; //车站名
    int ischange;        //是否为换乘站, 0-否, 1-换乘
};

struct weight {
    int wei;            //两个站间的权重, 即相差站数, 缺省为1
    int lno;            //两个顶点所在的线号
};

struct station BGvertex[MAXNUM]; //地铁网络图顶点数组
struct weight BGweights[MAXNUM][MAXNUM]; //网络图权重数组, 邻接矩阵
int Vnum = 0; //实际地铁总站数
```



问题7.1：主要代码实现

```
void initMap(){
    FILE *fp;
    int i, j, snum, lno, lnum, v1, v2;
    struct station st;
    ...
    fscanf(fp, "%d", &snum);
    for (i = 0; i < snum; i++) {
        fscanf(fp, "%d %d", &lno, &lnum);    v1 = v2 = -1;
        for (j = 0; j < lnum; j++) {
            fscanf(fp, "%s %d", st.sname, &st.ischange);
            v2 = addVertex(st); //将该站加到站信息数组中，返回其下标
            if (v1 != -1) {
                BGweights[v1][v2].wei = BGweights[v2][v1].wei = 1;
                BGweights[v1][v2].lno = BGweights[v2][v1].lno = lno;
            }
            v1 = v2;
        }
    }
    fclose(fp);
}
```



问题7.1：主要代码实现

```
void Dijkstra(int v0, int v1, int spath[]){
    int i, j, v, minweight;
    char wfound[MAXNUM] = {0}; //用于标记从v0到相应顶点是否找到最短路径, 0未找到, 1找到
    int sweight[MAXNUM] = {0};
    for (i = 0; i < Vnum; i++){
        sweight[i] = BGweights[v0][i].wei; //初始化数组Sweight
        spath[i] = v0; //初始化数组spath
    }
    sweight[v0] = 0;    wfound[v0] = 1;
    for (i = 0; i < Vnum - 1; i++) { //迭代VNUM-1次
        minweight = INFINITY;
        for (j = 0; j < Vnum; j++) //找到未标记的最小权重值顶点
            if (!wfound[j] && (sweight[j] < minweight)) {
                v = j; minweight = sweight[v]; }
        wfound[v] = 1; //标记该顶点为已找到最短路径
        if (v == v1) return; //找到(v0,v1)最短路径, 返回
        for (j = 0; j < Vnum; j++) //找到未标记顶点且其权值大于v的权值+(v,j)的权值, 更新其权值
            if (!wfound[j]&&(BGweights[v][j].lno>0)&&(minweight+BGweights[v][j].wei<sweight[j])){
                sweight[j] = minweight + BGweights[v][j].wei;
                spath[j] = v; //记录前驱顶点
            }
    }
}
```

思考一下如何计算多种换乘?
(即存在多个最短路径)



问题7.1：主要代码实现

```
void printPath(int v0, int v1, int spath[]){
    char path[80] = {0}, buf[80];
    //bcount为从v0到v1最短路径的乘坐站数，sc为乘坐某一线路的站数
    int board[80], bcount = 0, line = -1, sc = 0;
    int i;
    do { //获得乘坐站序列，该序列为倒的，即起始站在最后
        board[bcount++] = v1;
    } while ((v1 = spath[v1]) != v0);
    board[bcount++] = v0;
    line = BGweights[board[bcount - 1]][board[bcount - 2]].lno;
    sprintf(buf, "%s - %d(", BGvertex[board[bcount - 1]].sname, line);
    strcpy(path, buf);    sc = 1;
    for (i = bcount - 2; i > 0; i--, sc++)
        if (BGweights[board[i]][board[i - 1]].lno != line) {
            line = BGweights[board[i]][board[i - 1]].lno;
            sprintf(buf, "%d) - %s - %d(", sc, BGvertex[board[i]].sname, line);
            strcat(path, buf);    sc = 0;
        }
    sprintf(buf, "%d) - %s\n", sc, BGvertex[board[i]].sname);
    strcat(path, buf);
    puts(path);
}
```



提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题

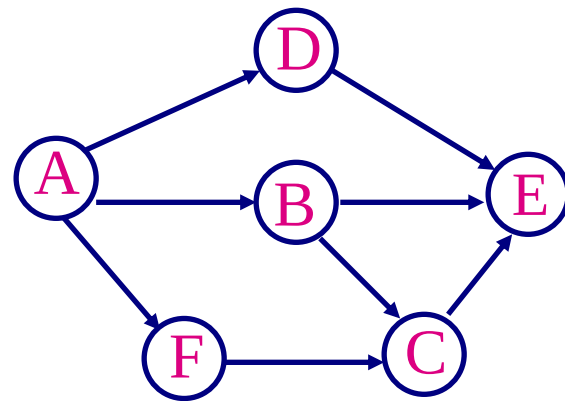


7.6 AOV网与拓扑排序*

AOV网的定义

以顶点表示活动，以有向边表示活动之间的优先关系的有向图称为**顶点表示活动的网**(Activity On Vertex Network), 简称**AOV网**

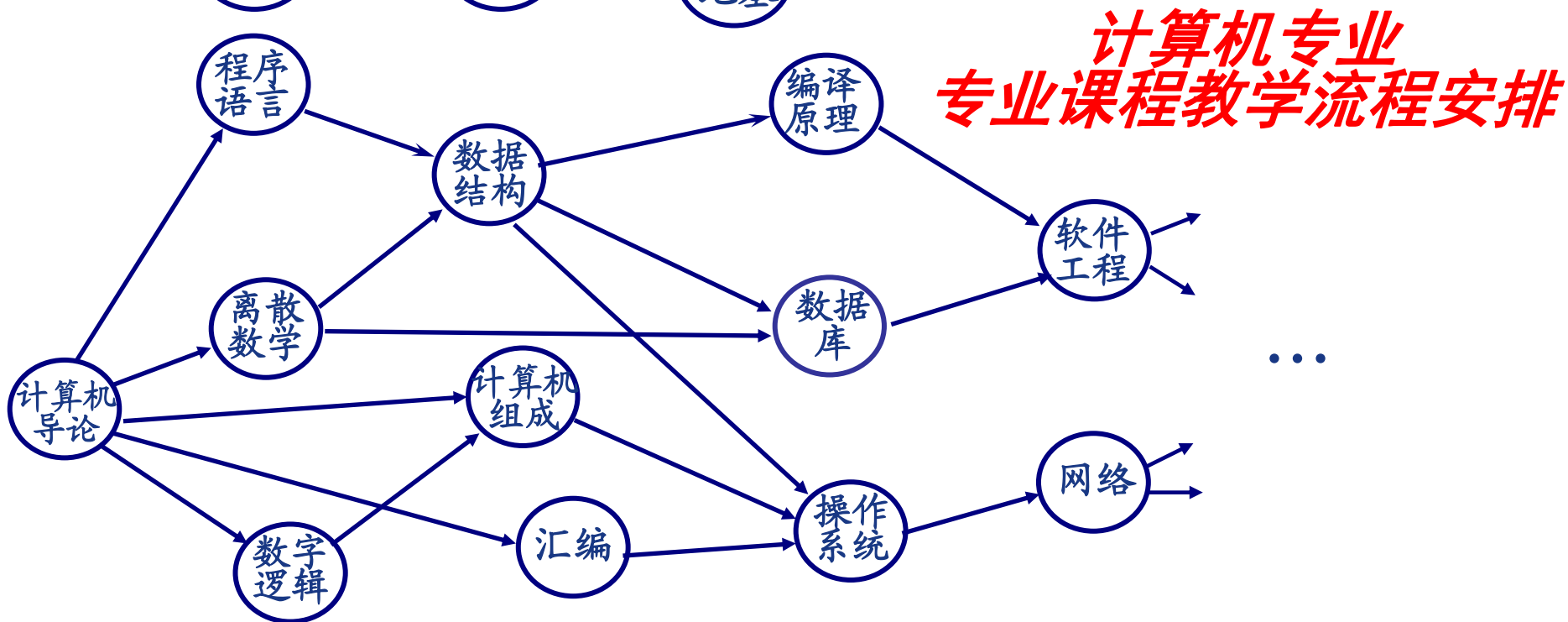
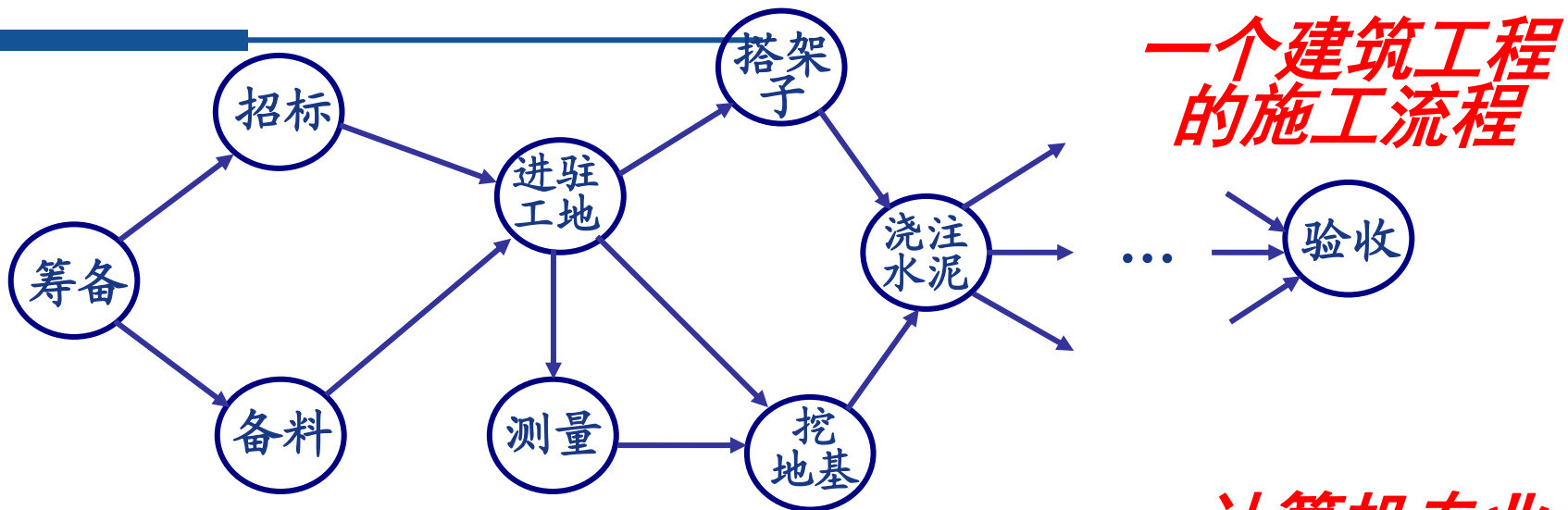
在AOV网中，若顶点 i 到顶点 j 之间有路径，则称顶点 i 为顶点 j 的前驱，顶点 j 为顶点 i 的后继；若顶点 i 到顶点 j 之间为一条有向边，则称顶点 i 为顶点 j 的直接前驱，顶点 j 为顶点 i 的直接后继



一个AOV网

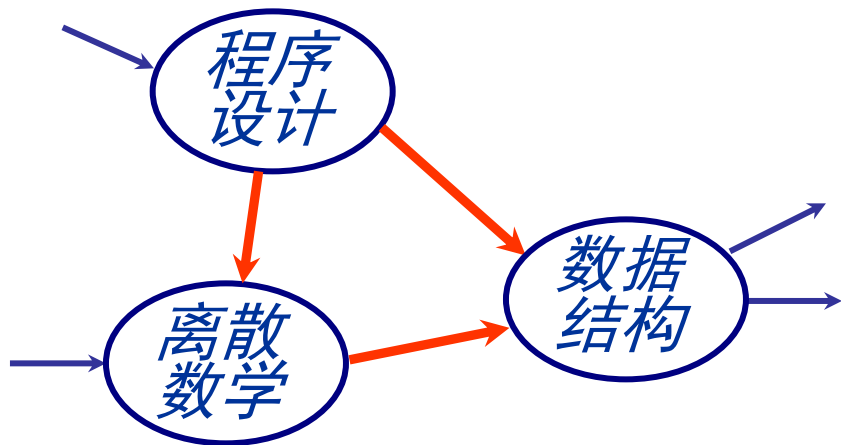


示例





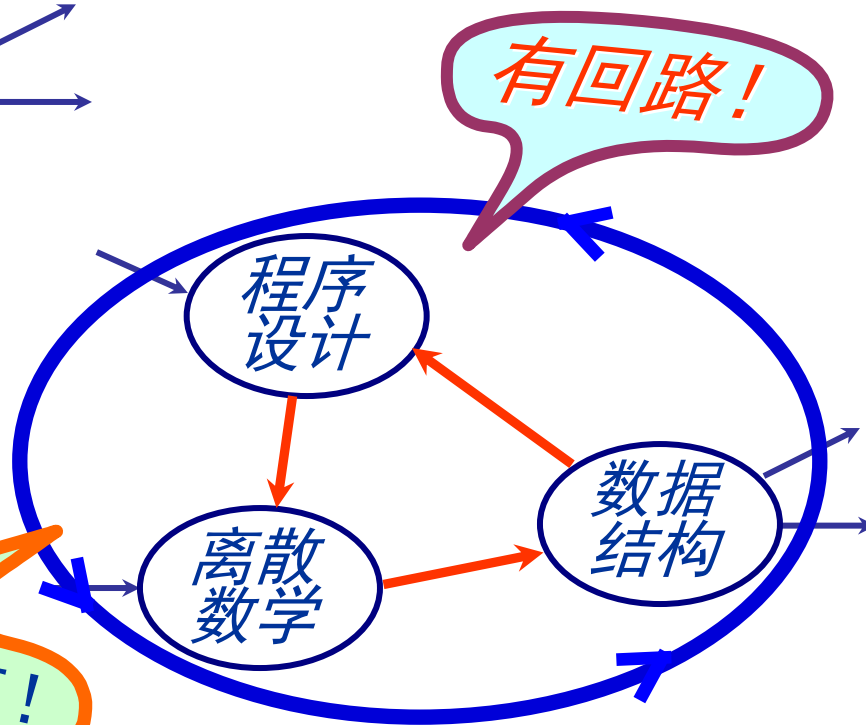
AOV: 工程能否正常进行



能够正常进行

没有入度为0的顶点!

特点



有回路!

不能够正常进行



拓扑排序

- ◆检测工程能否正常进行，首先要判断对应的AOV网中是否存在回路，达到该目的最有效的方法之一是对AOV网构造其顶点的拓扑序列，即对AOV网进行拓扑排序
- ◆由某个集合上的一个偏序得到该集合上的一个全序的操作称为**拓扑排序**

设 $G=(V,E)$ 是一个具有 n 个顶点的有向图， V 中的顶点序列 $v_1, v_2, \dots, v_n$ ，满足若从顶点 v_i 到 v_j 有一条路径，则在顶点序列中顶点 v_i 必在顶点 v_j 之前，则称这样的顶点序列为一个**拓扑序列**。构造拓扑序列的过程就是**拓扑排序**



拓扑排序

- ◆ 构造AOV网一个顶点序列，使得该顶点序列满足下列条件：
 - ✓1. 若在AOV网中,顶点 i 优先于顶点 j ,则在该序列中顶点 i 仍然优先于顶点 j ;
 - ✓2. 若在AOV网中， 顶点 i 与顶点 j 之间不存在优先关系,则在该序列中建立它们的优先关系,即顶点 i 优先于顶点 j ， 或者顶点 j 优先于顶点 i ;
 - ✓3. 若能构造出这样的拓扑序列， 则拓扑序列包含**AOV网的全部顶点**， 说明AOV网中没有回路

若构造不出这样的序列,
说明AOV网中存在回路



拓扑排序方法

入度为0!

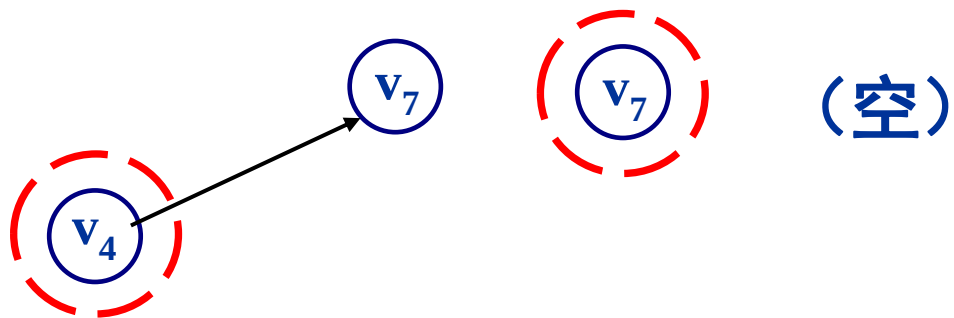
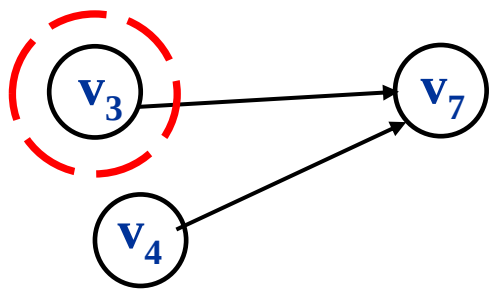
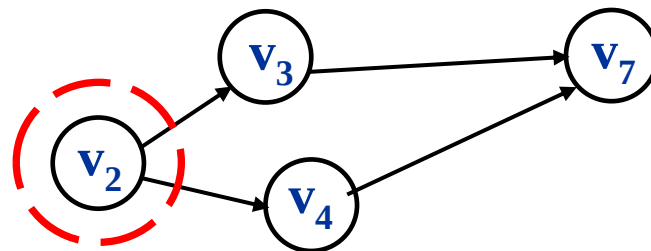
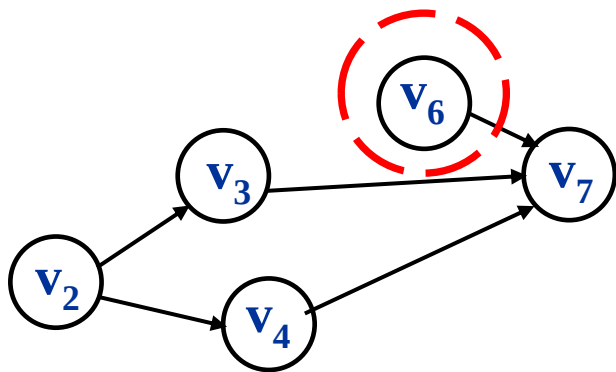
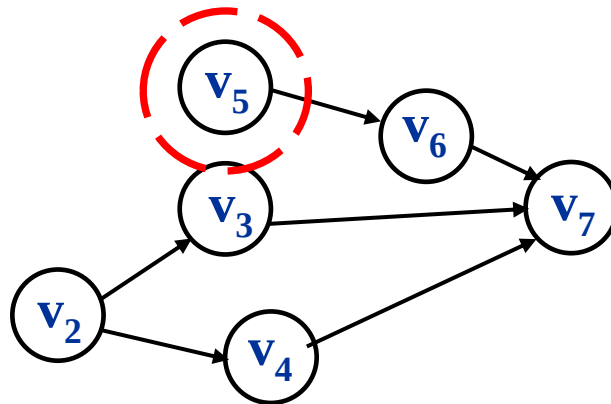
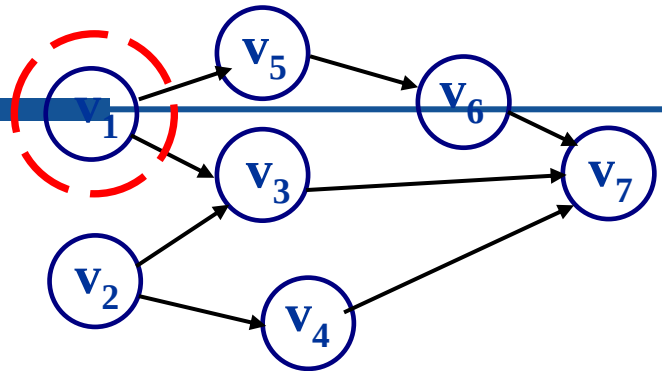
1. 从AOV网中任意选择一个没有前驱的顶点;
2. 从AOV网中去掉该顶点以及以该顶点为出发点的所有边;
3. 重复上述过程, 直到AOV网中的所有顶点都被去掉, 或者AOV网中还有顶点, 但不存在入度为0的顶点。

前者说明AOV网中无回路,
后者说明AOV网中存在回路。



示例

拓扑序列不一定唯一

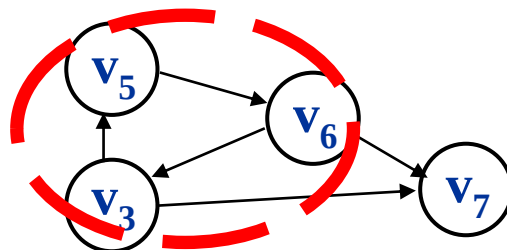
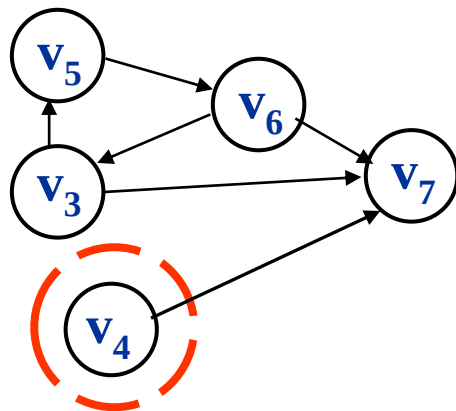
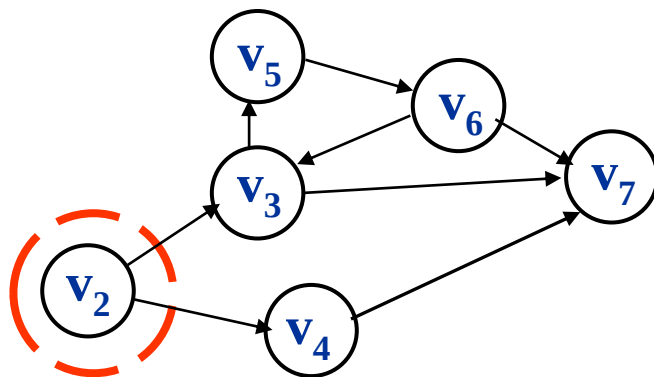
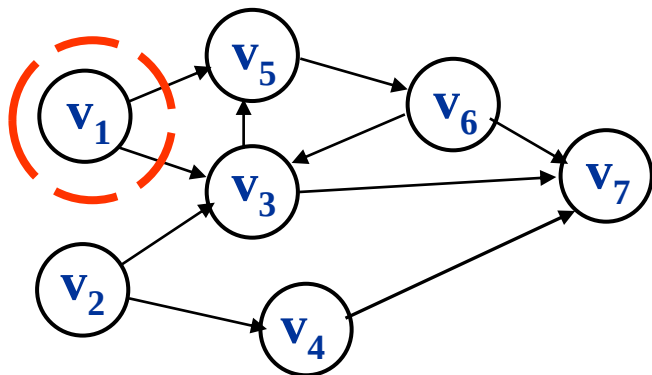


拓扑序列

$v_1 v_5 v_6 v_2 v_3 v_4 v_7$



示例



不存在拓扑序列
网中有回路!

$v_1 v_2 v_4$



算法实现

采用邻接表存储

1. 首先建立一个入度为0的顶点栈，将网中所有入度为0的顶点分别进栈
2. 当堆栈不空时，反复执行以下动作：
 - 从顶点栈中退出一个顶点，并输出它；
 - 从AOV网中删去该顶点以及以它发出的所有边，并分别将这些边的终点的入度减1；
 - 若此时边的终点的入度为0，则将该终点进栈；
3. 若输出的顶点个数少于AOV网中的顶点个数，则报告网中存在回路，否则，说明该网中不存在回路



思考?

除了进行拓扑排序，还可以
采用什么方法判断一个有向图
是否存在回路



进行**深度优先遍历**。若从某个顶点 v 出发，
遍历结束前出现了从顶点 u 到顶点 v 的回边，
则可以断定图中包含顶点 v 到顶点 u 的回路



提纲：图

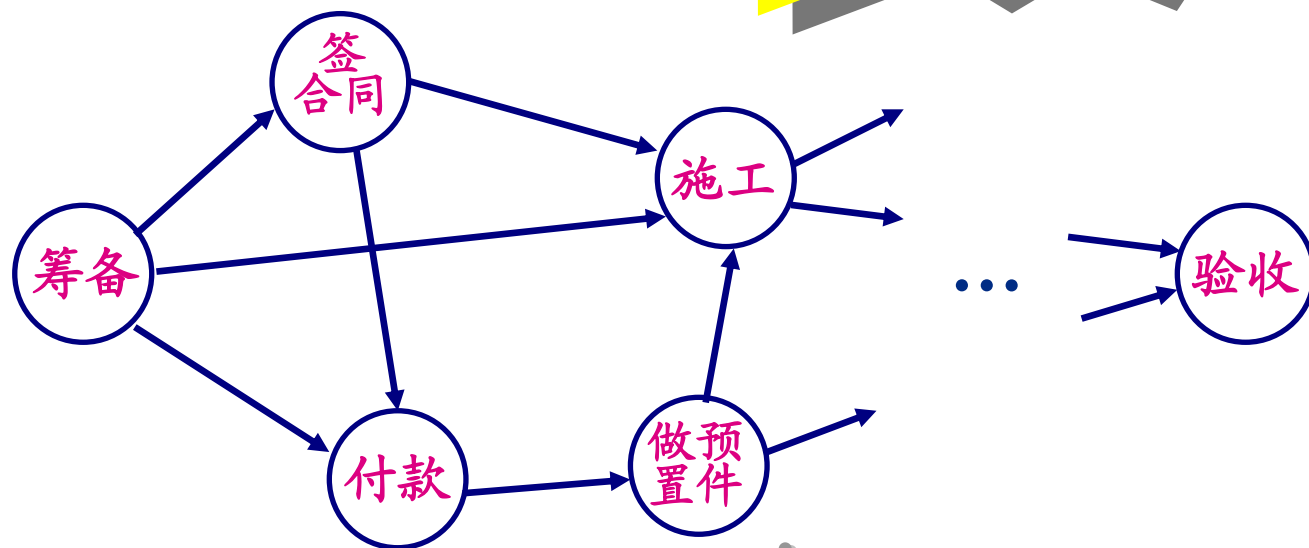
- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题



7.7 AOE网与关键路径*

更关心

1. 每个活动持续多少时间?
2. 完成整个工程至少需要多少时间?
3. 哪些活动是关键活动?

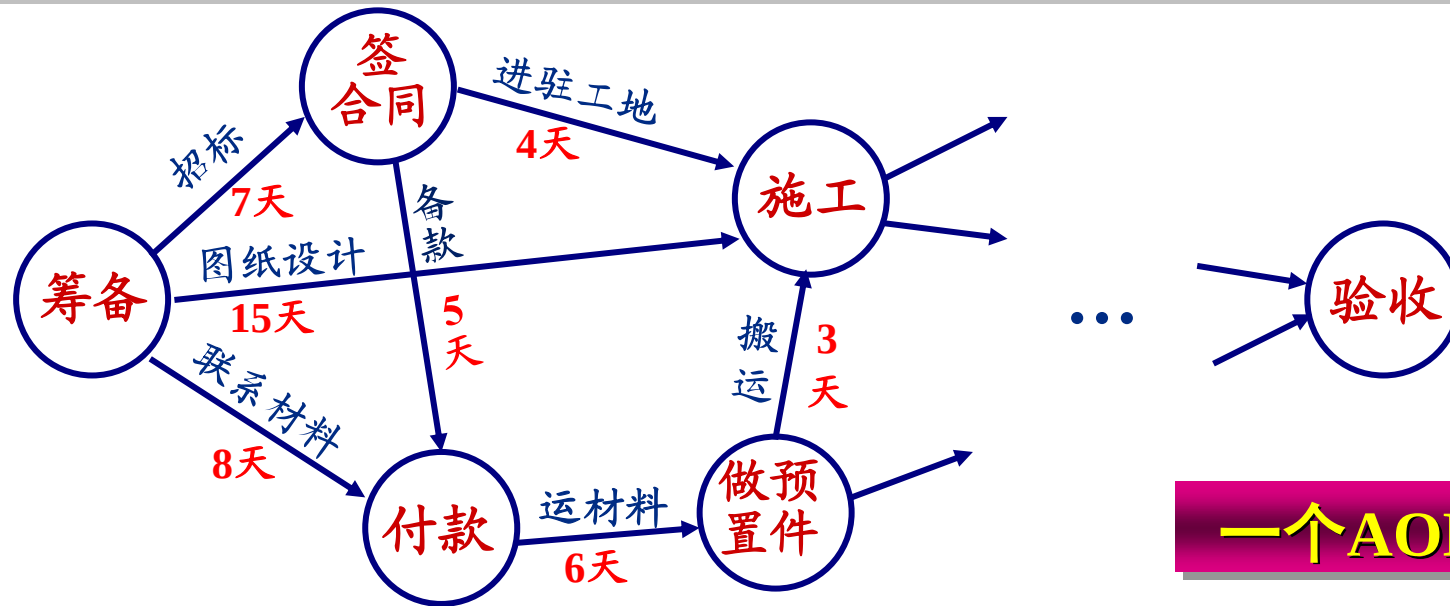


一个AOV网



一. AOE网的定义

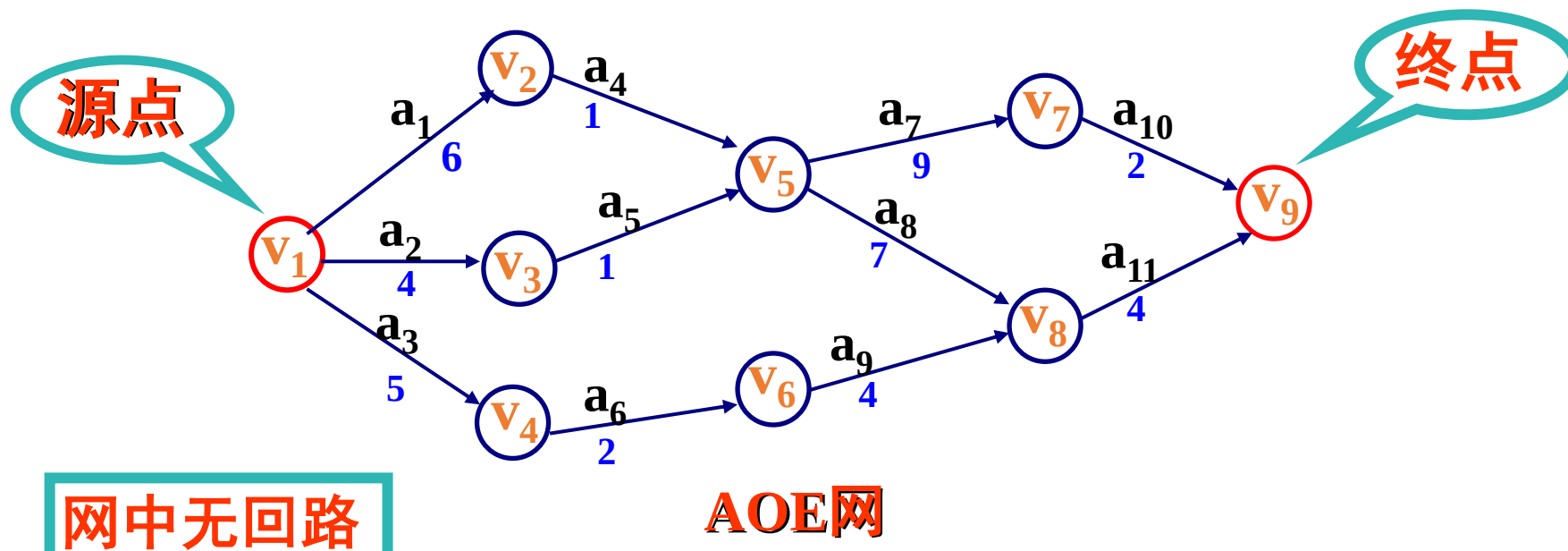
AOE(Activity On Edge)网为一个带权的有向无环图，其中，以顶点表示**事件**，有向边表示**活动**，边上的权值表示活动持续的**时间**



一个AOE网



示例：AOE网



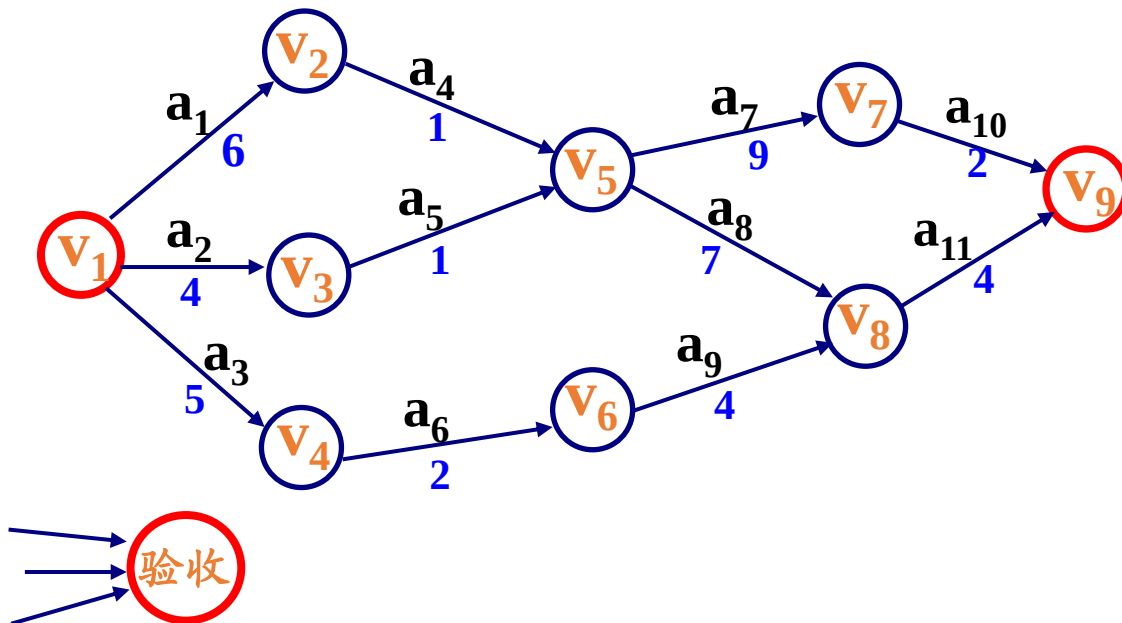
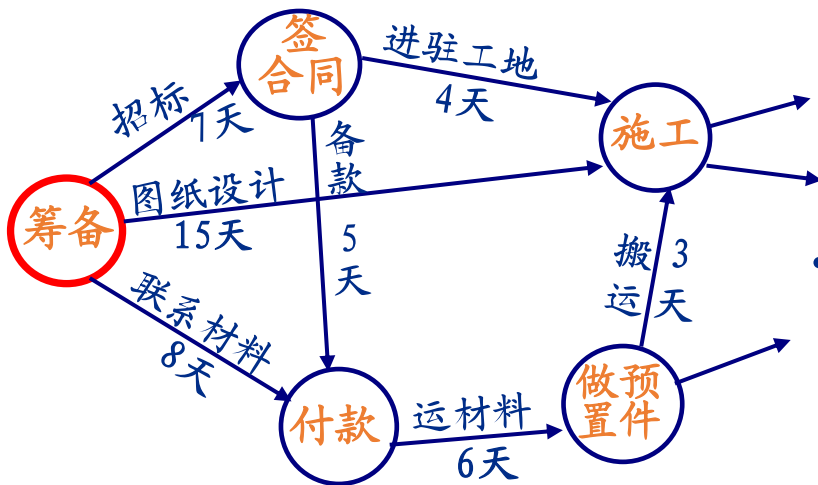
正常情况下，AOE网中只有一个入度为0的顶点，称之为源点；有一个出度为0的顶点，称之为终点



AOE网的特点

1. 只有在某个顶点所代表的事件发生以后, 该顶点引发的活动才能开始

2. 进入某事件的所有边代表的活动都已完成, 该顶点代表的事件才能发生





关键路径

二.AOE网的存储方法

采用邻接矩阵存储方法

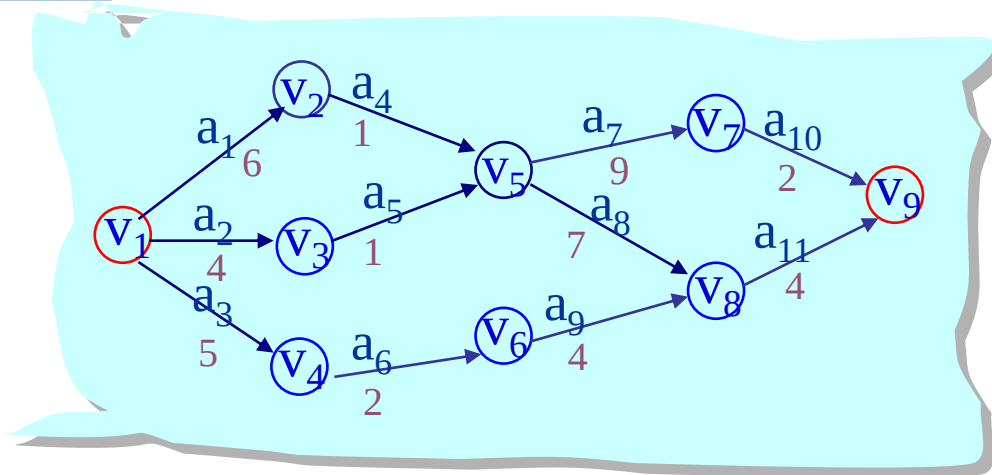
三.关键路径

1.关键路径的定义

从源点到终点的路径中具有最大长度的路径为关键路径；关键路径上的活动称为关键活动

2.关键路径的特点

- 1) 关键路径的长度(路径上的边的权值之和)为完成整个工程所需要的最短时间
- 2) 关键路径的长度变化(即任意关键活动的权值变化)将影响整个工程的进度，而其他非关键活动在一定范围内的变化不会影响工期





求关键活动的思路

缓冲时间/
松弛时间/
时间余量

$e[i]$ — 活动 a_i 的最早开始时间;
 $l[i]$ — 活动 a_i 的最晚开始时间;
若 $l[i]-e[i]=0$, 则说明活动 a_i 为一个关键活动。

$ee[k]$ — 事件 k 的最早发生时间

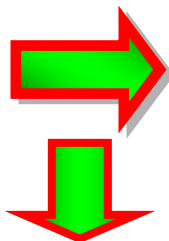


$le[k]$ — 事件 k 的最晚发生时间



结论

事件 k 的最早发生时间 $ee[k]$
事件 k 的最晚发生时间 $le[k]$



活动的 a_i 最早开始时间 $e[i]$
活动的 a_i 最晚开始时间 $l[i]$

a_i 为关键活动

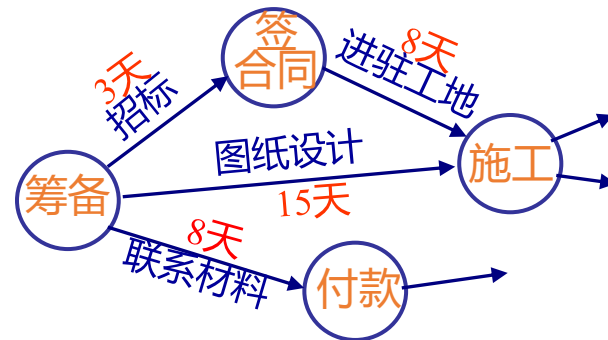
求 $e[i]=l[i]$



求关键路径

1. 计算事件k的最早发生时间 $ee[k]$

事件k的最早发生时间决定了由事件k出发的所有活动的最早开始时间;该时间是指从源点到顶点(事件)k的最大路径长度

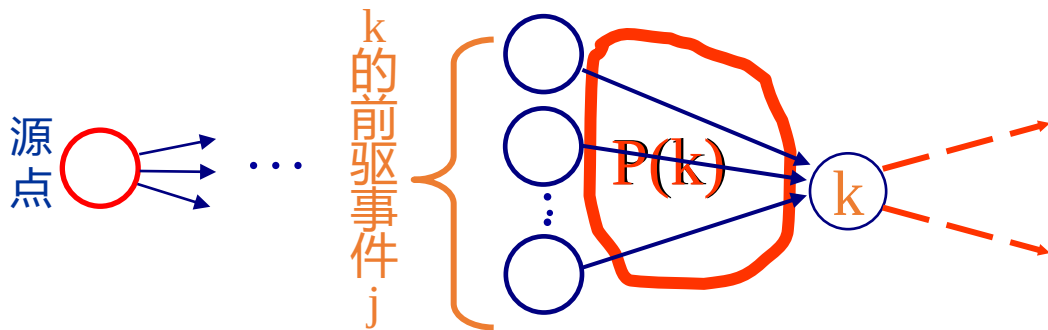


计算方法

$$ee[0] = 0$$

$$ee[k] = \text{MAX} \{ ee[j] + \langle j, k \rangle \text{的权} \}$$

$\langle j, k \rangle \in P(k)$

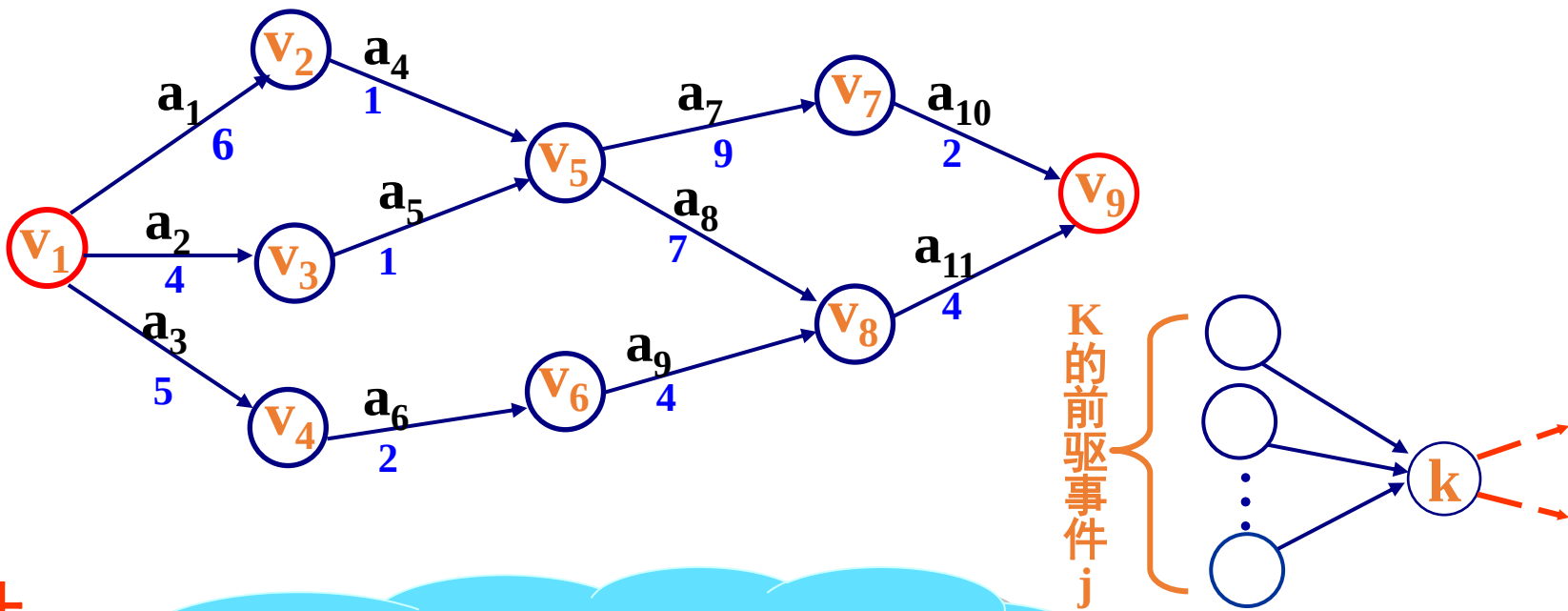




示例

ee[k] :

0	6	4	5	7	7	16	14	18
v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9



计算方法

$$ee[0] = 0$$

$$ee[k] = \text{MAX}\{ee[j] + \langle j, k \rangle \text{的权} \mid \langle j, k \rangle \in P(k)\}$$



求关键路径 (续)

2. 计算事件k的最晚发生时间 $le[k]$

所谓事件k的最晚发生时间是指不影响整个工期前提下事件k必须发生的最晚时间，它必须保证从事件k发出的所有活动的终点事件的最迟发生时间

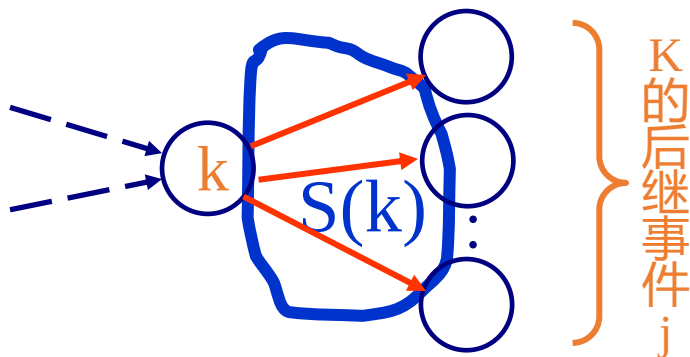
计算方法

$$le[n-1] = ee[n-1]$$

$$le[k] = \underset{\langle k,j \rangle \in S(k)}{\text{MIN}} \{ le[j] - \langle k,j \rangle \text{的权} \}$$

从后向前反推计算

k的后继事件



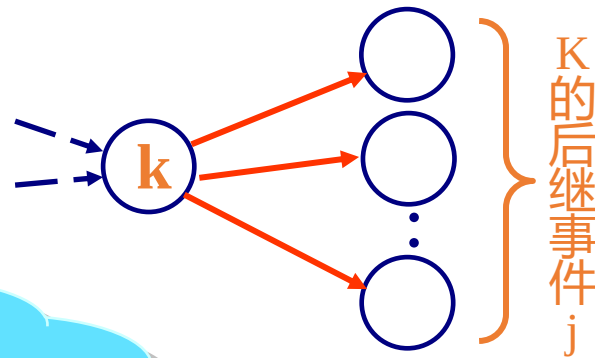
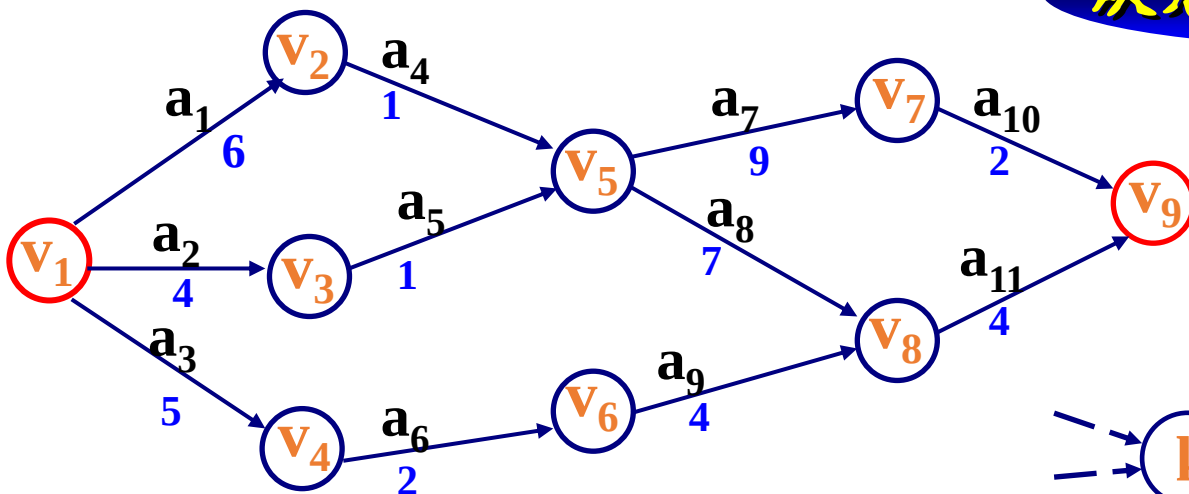


示例

le[k] :

0	6	6	8	7	10	16	14	18
v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9

从后向前反推计算



计算方法

$$\begin{aligned} le[n-1] &= ee[n-1] \\ le[k] &= \min_{\langle k,j \rangle \in S(k)} \{ le[j] - \langle k,j \rangle \text{的权} \} \end{aligned}$$



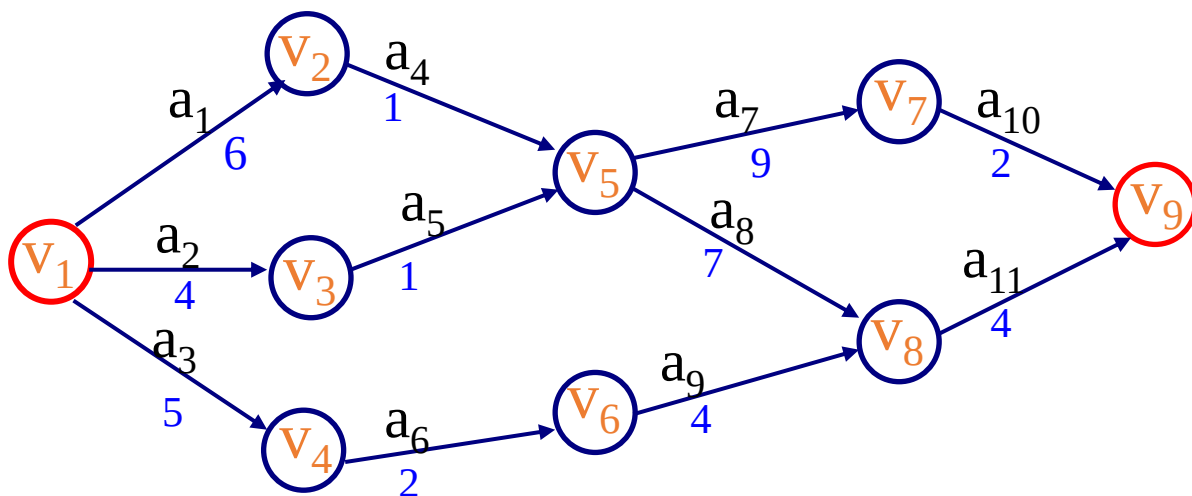
求关键路径 (续)

3. 计算活动 i 的最早开始时间 $e[i]$

所谓活动 i 的最早开始时间实际上是事件 k 发生的最早时间，即只有事件 k 发生，活动 i 才能开始



计算方法 $e[i] = ee[k]$





示例

$ee[k]$:

0	6	4	5	7	7	16	14	18
---	---	---	---	---	---	----	----	----

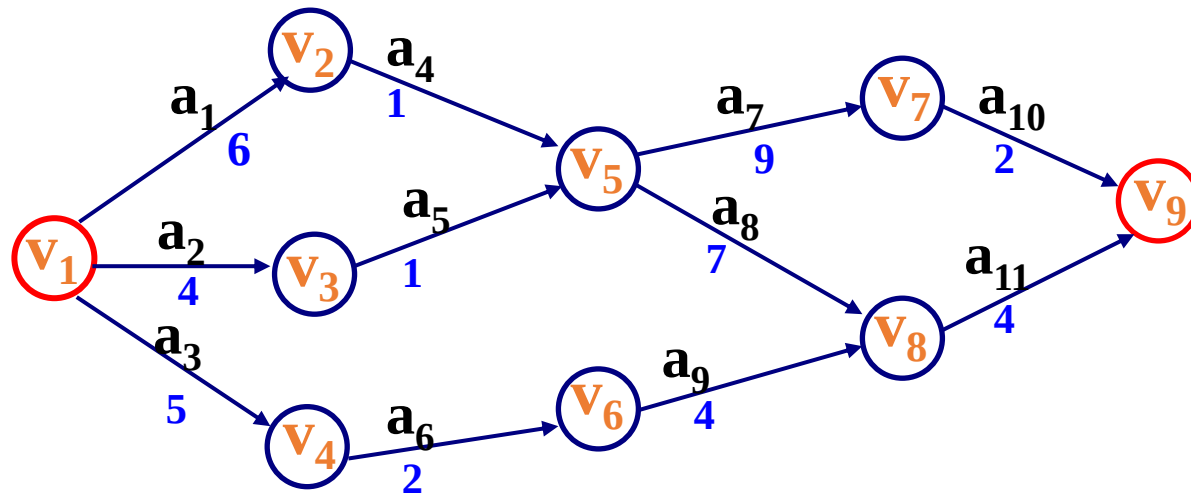
 (事件的最早发生时间)

$v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \ v_7 \ v_8 \ v_9$

$e[i]$:

0	0	0	6	4	5	7	7	7	16	14
---	---	---	---	---	---	---	---	---	----	----

$a_1 \ a_2 \ a_3 \ a_4 \ a_5 \ a_6 \ a_7 \ a_8 \ a_9 \ a_{10} \ a_{11}$



计算方法 $e[i] = ee[k]$



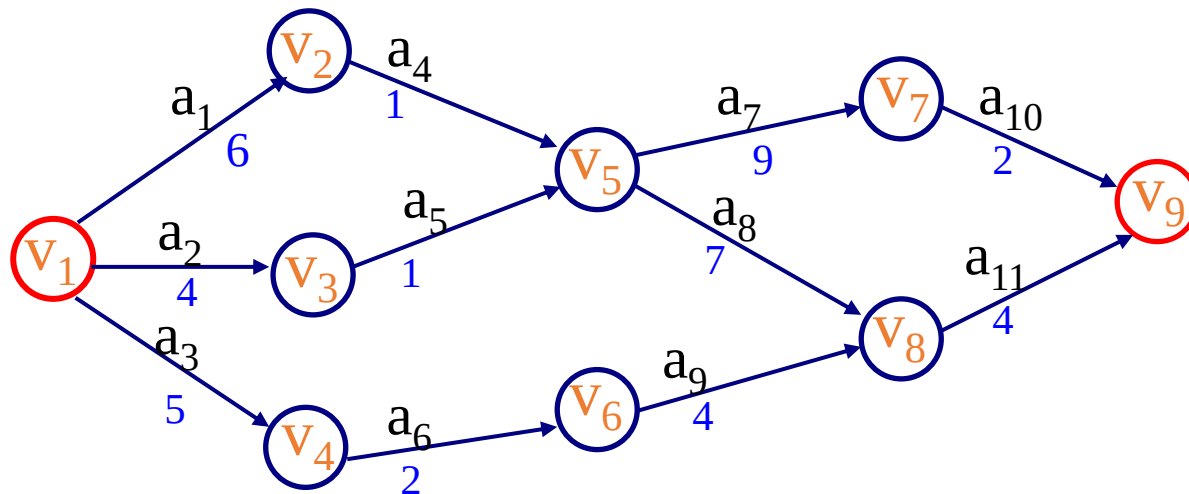
求关键路径 (续)

4. 计算活动 i 的最晚开始时间 $l[i]$

所谓活动 i 的最晚开始时间是指不推迟整个工期的前提下活动 i 开始的最晚时间



计算方法 $l[i] = le[j] - \langle k, j \rangle \text{的权}$





示例

$le[j]$:

0	6	6	8	7	10	16	14	18
---	---	---	---	---	----	----	----	----

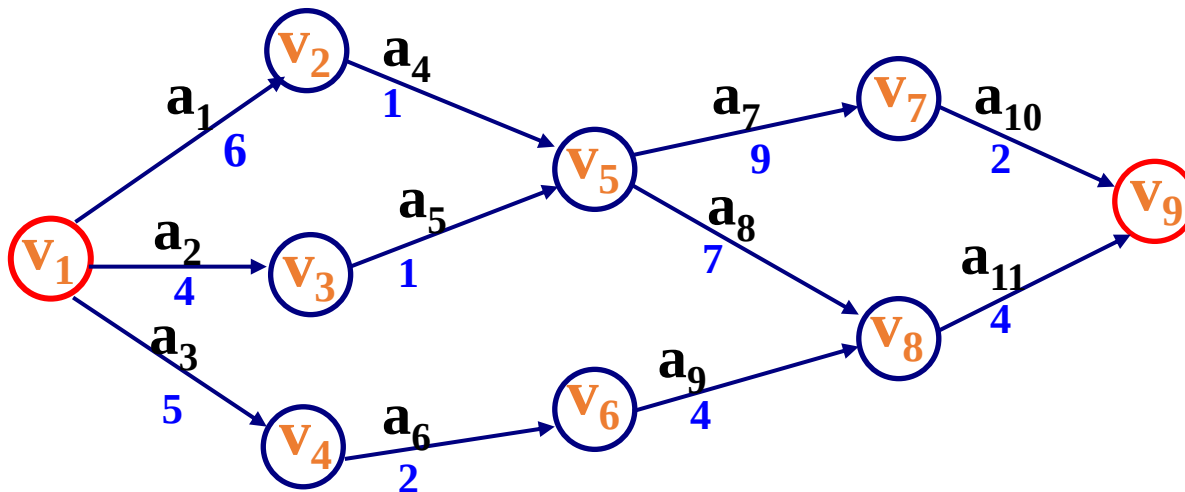
 (事件的最晚发生时间)

$v_1 \ v_2 \ v_3 \ v_4 \ v_5 \ v_6 \ v_7 \ v_8 \ v_9$

$l[i]$:

0	2	3	6	6	8	7	7	10	16	14
---	---	---	---	---	---	---	---	----	----	----

$a_1 \ a_2 \ a_3 \ a_4 \ a_5 \ a_6 \ a_7 \ a_8 \ a_9 \ a_{10} \ a_{11}$



计算方法 $l[i] = le[j] - \langle k, j \rangle \text{的权}$



求关键路径 (续)

5. 求出关键活动与关键路径

计算方法: $l[i] = e[i]$

$e[i]$:

0	0	0	6	4	5	7	7	7	16	14
0	1	2	3	4	5	6	7	8	9	10

$l[i]$:

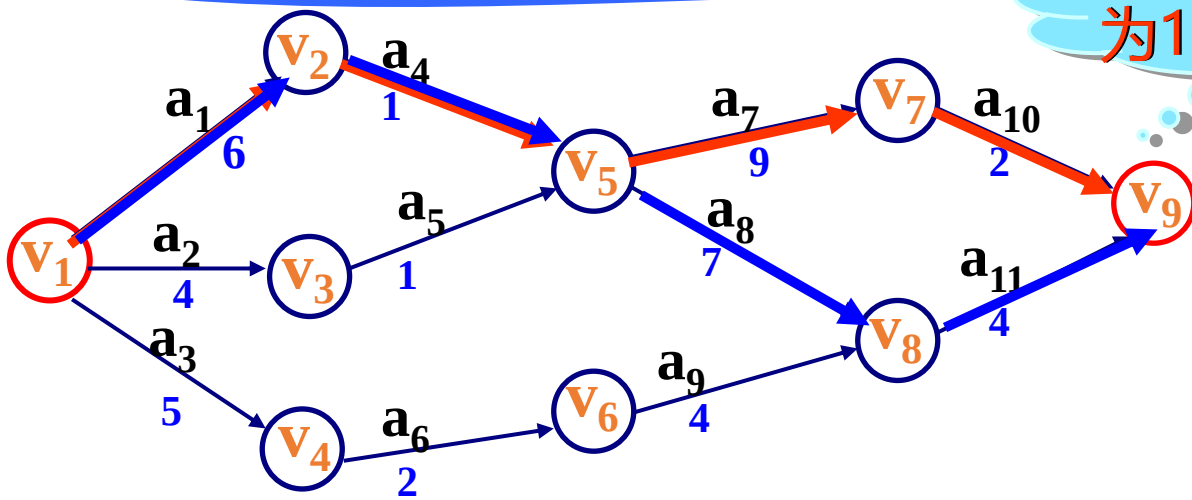
0	2	3	6	6	8	7	7	10	16	14
0	1	2	3	4	5	6	7	8	9	10

a_i 是关键活动

关键活动

a_1 a_4 a_7 a_8 a_{10} a_{11}

关键路径的长度为18个时间单位。





提纲：图

- ◆ 7.1 图的基本概念
- ◆ 7.2 图的存储结构
- ◆ 7.3 图的遍历
- ◆ 7.4 最小生成树
- ◆ 7.5 最短路径
- ◆ *7.6 AOV网与拓扑排序
- ◆ *7.7 AOE网与关键路径
- ◆ *7.8 网络流量问题



7.8 网络流量问题*

一. 相关概念

设给定边容量为 $c_{v,w}$ 的有向图 $G=(V,E)$ （容量可以表示通过一个管道的水、电、交通、网络等最大流量）。有两个顶点，一个是 s 称为源点(source)，一个是 t 称为汇点(sink)。对于任一条边 (v,w) ，最多有“流”的 $c_{v,w}$ 个单位（容量）可以通过。在既不是源点 s 又不是汇点 t 的任一顶点 v ，总的进入流必须等于总的发出的流。每条边上的流满足下面两个条件：

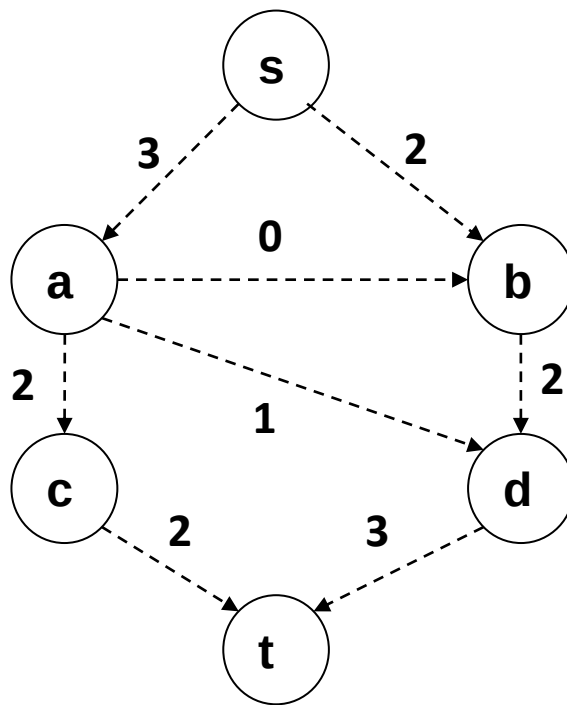
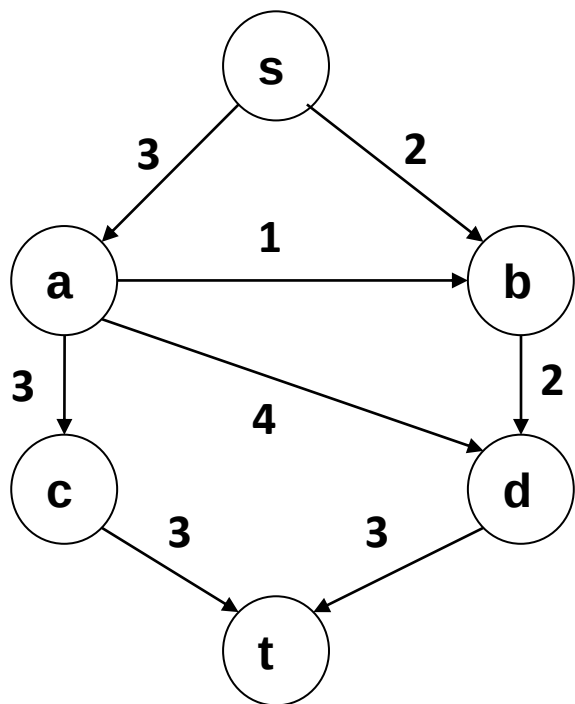
- ◆ 通过边的流不能大于边的容量（容量约束）
- ◆ 到达顶点 v 的流的总和与从 v 流出的总合相同，其中 v 不是源点或汇点。（流守恒）

最大流问题：确定从 s 到 t 可以通过的最大流量。



示例

对于下（左）图，其最大流是5，如右图所示





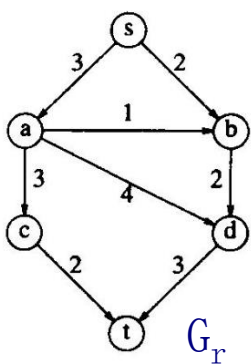
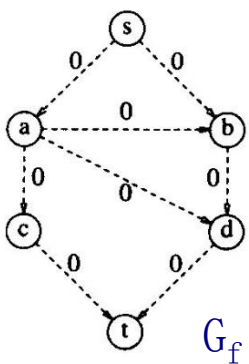
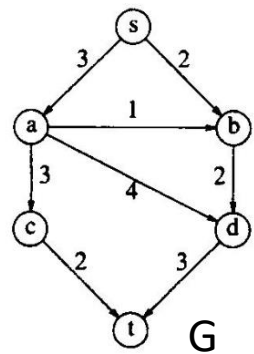
最大流量算法：Ford-Fulkerson算法

算法设有3个图（原图 G 、流图 G_f 、残余图 G_r ），在其上分阶段进行。 G_f 表示在算法的任意阶段已经达到的流，算法终止时其包含最大流； G_r 称为残余图（residual graph），它表示每条边还能再添加多少流（即还残余多少流），对于 G_r 中每条边（称为残余边，residual edge）可以从其容量中减去当前流来计算其残余流。

1. 初始时 G_f 所有边都没有流（流为0）， G_r 与 G 相同；
2. 每个阶段，先从 G_r 中找一条从 s 到 t 的路径（称为增长路径augmenting path）；
 - a. 将该路径上最小边的流量作为整个路径的流（权），并将路径加至流图 G_f 中；
 - b. 将该权值路径从 G_r 中减去，若某条边权值为0，则从 G_r 中除去；
 - c. 将具有该权的反向路径加到 G_r 中；
 - d. 重新执行步骤2，直到 G_r 中无从 s 到 t 的路径；
3. 将 G_f 中顶点 t 的每条入边流值相加得到最大流。



算法原理示例



原图 G 、流图 G_f 和残余图 G_r 初始状态

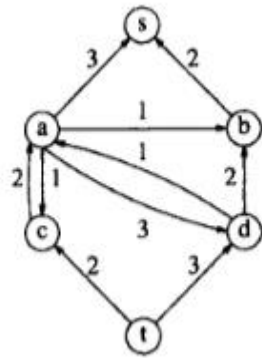
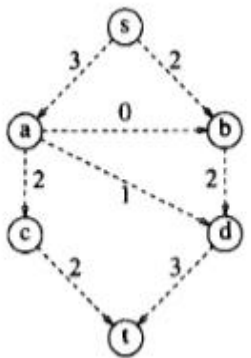
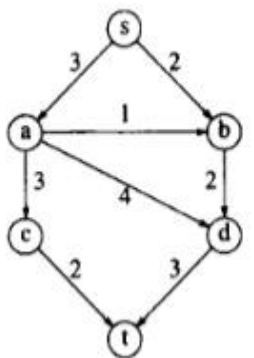
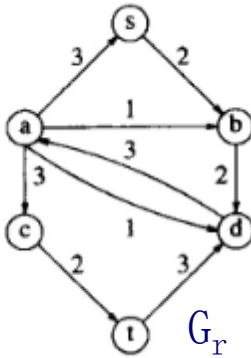
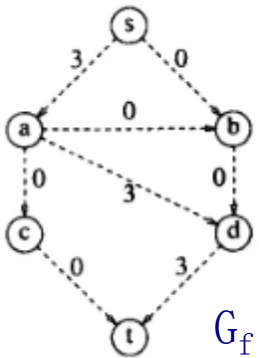
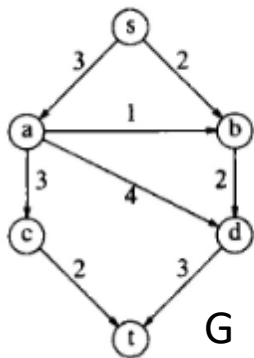
阶段一:

1. 在 G_r 中找到一条路径 $s-a-d-t$
2. 将该路径中最小边流3作为整个路径流, 并将该路径及其流加到 G_f 中
3. 在 G_r 中减去该路径及其流, 若某边流为0, 则从图中删除
4. 将该路径反向加到 G_r 中

阶段二:

1. 在 G_r 中找到一条路径 $s-b-d-a-c-t$
2. 将该路径中最小边流2作为整个路径流, 并将该路径及其流加到 G_f 中
3. 在 G_r 中减去该路径及其流, 若某边流为0, 则从图中删除
4. 将该路径反向加到 G_r 中

由于 G_r 中已没有从 s 至 t 的路径, 算法结束。此时 G_f 中 t 的入流之和即为**最大流**。



G

G_f

G_r



本章小结：图

◆图的基本概念

- ✓顶点和边
- ✓有向图、无向图、网
- ✓度、路径和路径长度
- ✓子图、图的连通、生成树

◆图的存储方法

- ✓邻接矩阵
- ✓邻接表

◆图的遍历

- ✓深度优先遍历 (DFS)
- ✓广度优先变量 (BFS)



本章小结：图（续）

◆最小生成树

- ✓生成树、最小生成树
- ✓Prim算法

◆最短路径问题

- ✓路径、路径的长度，单源最短路径
- ✓Dijkstra算法

◆图的应用*

- ✓AOV网与拓扑排序
- ✓AOE网与关键路径
- ✓网络流量问题